

Imperial College London

MSC ADVANCED COMPUTING INDIVIDUAL PROJECT

FINAL REPORT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Evaluating the Explanatory Properties of Large Language Models

Abstract

As LLMs become more and more mainstream, and take center stage in the very essence of our society, it is important to assess, interpret, and verify the content generated by these models, especially owing to their increasing usage in critical systems. This research investigates the explanatory properties of large language models (LLMs) through a comprehensive evaluation of various explanation generation strategies. It also proposes a novel faithfulness evaluation test that measures the alignment between explanations generated by models and their implicit reasoning and knowledge base. Explanations are generated in the form of connected reasoning chains and argumentation structures (based on formal argumentation theory), and subsequently evaluated using well-defined metrics that quantify their intrinsic properties. The proposed faithfulness test explores a new paradigm of evaluation among various open-access LLMs that ablates the generated explanations in the latent space, and examines the difference in prediction outcomes before and after this perturbation process. Explanations were generated on claims and questions sampled from numerous datasets such as TruthfulQA, StrategyQA, MedQA (and their claim verification counterparts), covering a range of reasoning domains. The LLMs used as part of this elaborate benchmarking activity belong to the Mistral, Qwen, and Llama families. Experimental results reveal that Qwen produces semantically rich explanations but this comes at the cost of redundancy. Llama and Mistral model performances vary largely across metrics and domains, and are discussed in detail. Additionally, Llama records high faithfulness measures, while Mistral and Qwen seem unfaithful. In the end, a preliminary user study suggests that human perceptions of LLM explanations somewhat reflect the results as seen in metric-based evaluations. Together, these findings provide both empirical insight and intuitive insight that help in advancing the broader goal of building interpretable and trustworthy LLMs.

Contents

1	Introduction	6
1.1	Motivations	6
1.2	Research Questions	7
1.3	Contributions	8
2	Background & Related Work	9
2.1	Chain-of-Thought (CoT) Prompting	9
2.2	Argumentative LLMs (argLLMs)	11
2.3	Explanation Properties & Evaluation Metrics	13
2.4	Understanding Faithfulness	15
3	Theoretical Foundations	18
3.1	Metric Definitions and Theory	18
3.1.1	Deductive Metrics	18
3.1.2	Argumentative Metrics	19
3.2	Faithfulness: Attention Perturbation Test	21
3.2.1	Conceptual Basis	21
3.2.2	Rationale Token Selection	22
3.2.3	Attention Perturbation Mechanism	23
3.2.4	Faithfulness Evaluation via Label Flips	23
4	Implementation	24
4.1	Overview	24
4.1.1	Datasets Used	24
4.1.2	Models Used	27
4.2	Chain-of-Thought (CoT) Pipeline	28
4.2.1	Explanation Generation	28
4.2.2	Reasoning Chain Construction using NLI	29
4.2.3	Evaluation using Deductive Metrics	30
4.2.4	Aggregation and Reporting	31
4.3	ArgLLM pipeline	31
4.3.1	Explanation Generation	32
4.3.2	Evaluation using Argumentative Metrics	33
4.4	The SHAKE test pipeline	35
4.4.1	Overview	35
4.4.2	Rationale Token Selection & Alignment	36
4.4.3	Attention Hooks & Perturbation	37
4.4.4	Decision & Scoring	38
4.4.5	End-to-End Testing	38
4.5	Pilot User Study	39
4.5.1	Design	40
5	Results & Evaluation	41
5.1	Chain-of-Thought (CoT) Pipeline	41
5.1.1	Claim Verification	41
5.1.2	Question Answering	43
5.2	ArgLLM pipeline	45

5.2.1	From Structural Argumentative Metrics	45
5.2.2	From Semantic Argumentative Metrics	46
5.3	The SHAKE test pipeline	48
5.4	Pilot User Study Results	51
6	Conclusions & Future Work	54
A	Paraphrase & Negation Tests for ArgLLM Nodes	59

List of Figures

2.1	Comparison between Standard Prompting & CoT Prompting over five different Commonsense Reasoning benchmarks [WWS ⁺ 23]	10
2.2	argLLM pipeline [FDG ⁺ 25]	12
2.3	Accuracy of three baselines (direct prompting, confidence estimation, CoT prompting) & four variations of ArgLLMs on claim verification tasks [FDG ⁺ 25]	13
2.4	This example from a Czech-to-English translation highlights how a model’s attention mechanism can be unfaithful; (a) [to the left] when translating the word <i>sto</i> into <i>century</i> , an attention heatmap shows the model is correctly focusing on the word <i>sto</i> ; (b) [to the right] however, it’s also possible for the model to generate the exact same word, <i>century</i> , even when the heatmap shows it is not paying any attention to <i>sto</i> [MKS21]	17
3.1	Example rationale token selection and alignment; model-declared rationale words aligned to sub-word indices; (a) ‘moon’ → tokens[1] (‘moon’); (b) ‘cheese’ → tokens[5:7] (‘che’, ‘ese’)	22
4.1	High-level overview of the project implementation pipeline, from datasets to explanation generation, evaluation, and reporting.	25
4.2	End-to-end SHAKE pipeline (generalized for model’s last L attention layers)	36
5.1	Metric-wise plots for Mistral, Llama, and Qwen models: Claim Verification	42
5.2	Metric-wise heatmaps for Mistral, Llama, and Qwen explanations: Claim Verification	43
5.3	Metric-wise plots for Mistral, Llama, and Qwen explanations: QA	44
5.4	Metric-wise heatmaps for Mistral, Llama, and Qwen explanations: QA	45
5.5	Metric-wise plots for Mistral, Llama, and Qwen argLLM explanations: Claim Verification	47
5.6	Metric-wise plots for Mistral, Llama, and Qwen argLLM explanations: QA	48
5.7	Faithfulness measure, aka, flip percentage resulting from the novel attention perturbation: SHAKE test	50
5.8	Distribution of attention weights over different tokens before and after the perturbation: zeroing out and renormalization	52

List of Tables

4.1	Key hyperparameters and I/O conventions used in the CoT pipeline	30
4.2	Schema for user study data export	40
5.1	CoT explanation metrics on claim verification datasets. Arrows indicate preferred direction. Best per-row values are bold	42
5.2	Prediction accuracy of CoT explanations across datasets and models.	43
5.3	CoT explanation metrics on QA datasets. Arrows indicate preferred direction. Best per-row values are bold	44
5.4	Claim-verification (argLLM) semantic metrics. Arrows indicate preferred direction. Best per-row values are bold	46
5.5	Prediction accuracy of argLLM explanations across claim verification datasets and models.	47
5.6	QA (argLLM) semantic metrics. Arrows indicate preferred direction. Best per-row values are bold	48
5.7	SHAKE test flip summary across datasets and models; flip % = no. of flipped Samples / final samples; (weighted) average flip rates by model: Mistral - 13.84%, Qwen - 10.48%, Llama - 86.37%	49
5.8	SHAKE example summary (TruthfulClaim, Llama)	49
5.9	Last-layer attention weights before/after perturbation. Targeted rows are the exact token indices mapped from the rationale tokens (“cannabis”, “personal”).	51
5.10	Per-participant summary of buy rates and average confidence/clarity.	53
5.11	Per-item responses mapped to dataset, model, and method (all participants).	53
A.1	Preliminary AKF results across datasets, models, and node polarities. PC = Paraphrase Consistency, NS = Negation Sanity, PS = Paraphrase Stability. Scores closer to 1 indicate stronger consistency or stability.	61

Chapter 1

Introduction

The coming of LLMs has catalyzed a huge shift in how we think about artificial intelligence and agentic systems. This is futuristic technology; in the form that we have aspired to develop for so long, in the form that has been popularized by pop culture for decades, it has finally arrived! It is no longer "coming soon." It is already here, at our fingertips, and it is here to stay. Today, these large language models that seem to have crept into almost all aspects of our lives stand on the verge of being massively incorporated into what we call "critical systems" - those that uphold the very fabric of human society. These include healthcare, education, law, defense, governance, and communication, among others.

1.1 Motivations

However, this large-scale and aggressive adoption of these AI models raises a pertinent concern: the lack of interpretability and transparency. LLMs are "black-box" models. [DS21] defines black-box models as "...systems that are viewed in terms of their inputs and outputs without any knowledge of their internal mechanism." Meaning, their internal workings are abstracted away from end-users, who interact with the models only in terms of their respective inputs and outputs. This leads us to question the veracity of their claims and highlights the need to make our artificially intelligent systems more explainable and trustworthy.

This lack of transparency affects the overall usability of these "critical systems", such that it hinders error detection & diagnosis, and introduces an undue element of uncertainty. For example, if LLMs are used to extract crucial information from patient data, unexplained errors can have catastrophic effects. Furthermore, responses might be plausible-sounding but may not actually reflect the true reasoning abilities and knowledge acquired by the model, inculcating a false sense of reliability. In other words, it is not enough that the model sounds correct, it should also BE correct.

Previous works delve deeper into studying the general reasoning structures behind LLM responses. Over the years, various methods have been developed to elicit explanations from LLMs about their answers and conclusions. One such technique that falls under the general paradigm of "instructional training" [WBZ⁺22] is the very famous Chain-of-Thought (CoT) prompting method [WWS⁺23]. It refers to the act of providing brief but logical guidance to the model to think rationally or in a certain way, and to sneak-peek into their decision-making processes that eventually lead to the prompt responses that we so readily desire. In other words, it is the virtual hand-holding of the LLM to make it think out loud the series of structured decisions it takes in order to reach a certain conclusion in response to user queries and prompts. In another method, "argLLMs" (short for Argumentative LLMs) define a framework to structure LLM answers such that they are not only explainable but also largely contestable [FDG⁺25]. This means that users can both, inspect and argue with the LLM on their internal reasoning logic when answering a particular question. It is a graphical network, a debate tree that allows for improved interpretability and understanding rather than compelling users to accept answers at their face value.

Despite the existence of such intricate explanation paradigms, the evaluation of these explanations still poses a significant challenge. Many previous works focus on human expert-annotated, manual evaluation strategies. A work from 2023 [BKPW23] evaluates LLM-generated explanations using a content analysis approach where expert academic instructors tagged each explanation as correct, incorrect, or missing based on its usefulness in identifying logical bugs in student-written code. In another paper [BDM⁺24], models were treated as surrogate participants to mimic a user study. Models performed the same tasks as the humans did in the study and the results were compiled & evaluated using the same two-way ANOVA ([Hen15], [All25a], [All25b]) and mean comparison tests used to assess the original human user study. Therefore, there is limited literature available that explore methods to computationally evaluate the internal properties of explanation. Additionally, there is a lack of enough performance benchmarks that signify explanation qualities, covering a range of large language models and datasets for different use cases. These research gaps form the very basis of this study.

More specifically, we aim to look at paradigm-specific evaluation metrics as defined in [KT24]. These demonstrate the explanation’s internal logical consistency ([ZYL⁺25], [TSO⁺25], [YCS⁺25], [WLAB25]), repetitiveness, relevance, and contestability. The idea is to implement an evaluation framework that accepts explanations from multiple paradigms and computes well-defined metrics that are interpretable. We also study a number of works that explore the concept of faithfulness in LLM explanation. Consequently, we attempt to propose a new evaluation test that combines multiple views and interpretations of said faithfulness.

1.2 Research Questions

The primary objective of this research is to determine whether explanations produced by different paradigms are actually coherent, logical, and useful for us users, and to do so with metrics that are executable and interpretable across tasks and models. As we define our research objectives, we look at the following questions.

RQ1: Do the explanations generated by the two methods: CoT and argLLMs satisfy the fundamental properties that make an explanation viable, as defined in [KT24]? If yes, then to what extent are these properties satisfied? And are these metrics comparable across different use cases? More specifically, for CoT, we examine Weak Relevance, Strong Relevance, and Redundancy as primary markers of internal validity. On the other hand, for argLLMs, we use Acceptability and Dialectical Non-Circularity as primary markers that the nodes in an argument graph are useful. This research theme requires an in-depth study of the nature and structure of the explanations generated by different models, using each of the methods used.

RQ2: Can we devise a new LLM faithfulness measure or evaluation test that can combine internal attribution ([LLG⁺22], [JW19], [WP19]) and semantic causality ([LLG⁺22], [ACL⁺23])? Existing measures in this area are uni-directional; they are either influenced by altering external factors like prompt semantics or probing into the internal layers of the model to understand the explanations. But there is no measure that combines these different aspects of faithfulness. We ask, do the reasons that models themselves identify as influential in their explanations actually drive the model’s predictions based on its internal workings? In other words, is a plausible-sounding explanation actually true? This RQ requires us to deep-dive into the basics of large language models and its underlying transformer architecture.

RQ3: Are the computed metrics in line with common human perception? As has been the norm in evaluating LLM explanations, a well-defined user study can help understand the difference between human judgment and mathematical evaluation of explanation types. It would be extremely valuable to find out whether the properties defined in the investigation capture the quality that end users actually value. Consequently, any evidence of alignment between these quantitative metrics and qualitative properties would further strengthen the impact of this research.

Together, these poignant research questions motivate us to conduct elaborate and complex experiments on different LLMs, and lead us to make the prominent contributions to the ever-evolving field of XAI.

1.3 Contributions

This study makes a number of interlinked contributions in the field of Explainable AI, and delivers a nuanced, comprehensive evaluation framework that examines LLM explanations generated for a number of claim verification and question answering datasets.

Firstly, we devise a comprehensive evaluation suite that accepts explanations in the form of reasoning chains, as well as argument graphs, and scores them on the basis of their intrinsic properties. The reasoning generated by the Chain-of-Thought prompting method is subjected to evaluation against metrics such as Weak Relevance, Strong Relevance, and Redundancy. On the other hand, the relationship between individual arguments, as generated by argumentative LLMs are examined using metrics like Acceptability and Non-Circularity. The architecture is modular and systematically employs these different evaluation metrics depending on the paradigm to which the generated explanation fundamentally belongs. The scripts used to generate these explanations on a large scale are reusable and are applied to multiple datasets that cover different domains. They represent various experimental conditions specific to each explanation generation technique "few-shot" CoT and 1-dimensional argLLM.

In order to thoroughly evaluate a large number of LLM explanations from different domains, it was extremely important to formalize and execute the implementations of the aforementioned metrics initially defined in [KT24]. Each of these descriptive metrics were implemented to fit the level of complexity produced by the explanations such that final evaluation scripts can directly call the relevant functions and calculate metrics on a per-dataset, per-model basis. In the upcoming sections, we have extensively discussed the qualitative basis and the mathematical foundations for these metrics. They clearly highlight the underlying idea and logic used to check and report the overall usability and veracity of the explanations.

Another major contribution of this work is a novel faithfulness evaluation test that assesses whether the explanations generated by models are actually true to their internal knowledge and reasoning abilities. We propose a "SHAKE" test, that probes into the internal attention module of the last layer(s) of the large language model and verifies their alignment with the output response generated. At first, the approach elicits explanation or "rationale" tokens from the model. These are words which the LLM says have been influential in its decision-making for label prediction. Then, we zero-out the model's attention to these tokens, and pass the pertinent claim through the pipeline once again to generate a binary label. By comparing the outcomes before and after this "perturbation", the method quantifies the binary label flip rate as a measure for quantifying both attributional ([LLG⁺22], [JW19], [WP19]) and causal ([LLG⁺22], [ACL⁺23]) faithfulness of the explanations.

Another one of our research objectives was to understand and study the alignment of these computed metrics with human judgment ([BKPW23], [BDM⁺24]). It is imperative to understand whether the explanations generated by these all-important large language models satisfy the general needs of users. Therefore, we conducted a preliminary user study to test whether the metrics of our research match the usefulness of LLM explanations as perceived by users. Frequent users from different domains rated explanations on their completeness, usefulness, and trustworthiness. The responses were anonymized to ensure a fair and valid initial study. This human assessment helps us understand whether the proposed metrics capture qualities that end-users actually value, and lays the groundwork for an extensive study, with a significantly large number of users.

These contributions deliver both theoretical basis and practical impact. The multi-dataset, multi-model benchmark using formal metric definitions provides the foundation for a unified evaluation of diverse reasoning tasks as performed by LLMs. The claim verification datasets such as TruthfulClaim, StrategyClaim, MedClaim allow a diverse perspective on the models' abilities to elaborate on its decision-making processes. We work with a number of open access models such as those from the Mistral, Llama, Qwen families, and report statistical results backed with representative cases and examples to highlight the effectiveness and authenticity of our research. The extensive faithfulness probes provide definitive context and allow explanation assessment based on internal model architectures and causality. Finally, the pilot user study ensures that the work remains relevant to the practical usage of LLMs on a day-to-day basis. With this combination, we take a step towards the development of more transparent, verifiable, and trustworthy AI systems that can be systematically used in today's critical system architectures.

Chapter 2

Background & Related Work

In this chapter, we discuss the theoretical and conceptual foundations of our experiments. We will first look at the two explanation generation methods: CoT prompting and argLLMs. We will discuss their history, the underlying math, pertinent use cases, and a number of useful variants that could come in handy during future extensions of our work. Next, we will try to understand the mathematical foundations behind the evaluation metrics that were used to examine and score the explanations. Furthermore, it will be worth taking a closer look at how these can be applied across the different explanation paradigms discussed. Finally, we will look at the all-famous faithfulness metric, its different interpretations across the available literature, and the gaps in its existing research. We will contrast the idea of surface-level plausibility with a deeper, more intricate reflection of the model’s knowledge. This will serve as the basis and motivation for our novel faithfulness evaluation test which we will discuss in further sections.

2.1 Chain-of-Thought (CoT) Prompting

Chain-of-Thought prompting stands tall as a major development in the field of Explainable AI (XAI). As described in [WWS⁺23], it provides a transparent, step-by-step procedure to inspect the reasoning employed by LLMs, rather than simply accepting given answers on their face value. The method falls into the broad range of methods that we usually call instructional training [WBZ⁺22], where we effectively instruct the model to think deeply about the solution and often, think in a certain way. The series of reasoning steps generated provide a way for users to assess the logic and rationale behind the responses and potentially critique them, thereby enhancing the interpretability and usefulness of these black-box models.

Basically, the method compels the models to "print" the intermediate reasoning steps of a task as natural language text before concluding on a final answer. In other words, it means prompting the model to generate a sequence of steps or "propositions" in addition to the final answer, decision, or prediction label. Internally, the model decomposes the original task or prompt into a series of sub-tasks and attempts to solve each one separately, leading to responses in the form of intermediate steps. Formally, an explanation E generated using CoT prompting can be viewed as an ordered list of propositions p_i :

$$E = \langle p_1, p_2, \dots, p_k \rangle, \quad \text{with conclusion } c.$$

Ideally, in the case of a vanilla CoT explanation, the propositions are in order, such that a proposition logically follows its predecessor. This results in a structure of connected propositions that resembles a chain. However, CoT explanations fall under the broad umbrella of deductive explanations [KT24] and can be represented as a matrix of propositions, with multiple propositions branching out from one. This leads to a structure that is more complex than a simple chain, even though a chain-like structure can still be found at its very core. This adjacency matrix of propositions can be computed using Natural Language Inference (NLI) and is imperative for evaluating CoT explanations. We will discuss this in further detail in the subsequent chapters.

The method, of course, has its variants: Zero-shot, One-shot, and Few-shot techniques, each more structured and detailed than its prior. However, the key difference between these methods

is the presence & the number of representative examples used to compel the model to reason in a particular direction. While the one-shot method uses only one exemplar to explain to the model how it wants the reasoning to proceed, few-shot methods provide multiple such "training" examples. As the name suggests, the zero-shot variant does not provide any examples to convey the desired thought pattern. Instead, it annotates the original prompt with the following statement: *"Let's think step-by-step"*. In the end, all variants are geared towards one objective, that is, to extract the hidden rationality behind LLM reasoning by explicitly prompting it.

		CSQA		StrategyQA		Date		Sports		SayCan	
Model		standard	CoT	standard	CoT	standard	CoT	standard	CoT	standard	CoT
UL2	20B	34.2	51.4	59.0	53.3	13.5	14.0	57.9	65.3	20.0	41.7
LaMDA	420M	20.1	19.2	46.4	24.9	1.9	1.6	50.0	49.7	7.5	7.5
	2B	20.2	19.6	52.6	45.2	8.0	6.8	49.3	57.5	8.3	8.3
	8B	19.0	20.3	54.1	46.8	9.5	5.4	50.0	52.1	28.3	33.3
	68B	37.0	44.1	59.6	62.2	15.5	18.6	55.2	77.5	35.0	42.5
	137B	53.6	57.9	62.4	65.4	21.5	26.8	59.5	85.8	43.3	46.6
GPT	350M	14.7	15.2	20.6	0.9	4.3	0.9	33.8	41.6	12.5	0.8
	1.3B	12.0	19.2	45.8	35.7	4.0	1.4	0.0	26.9	20.8	9.2
	6.7B	19.0	24.0	53.6	50.0	8.9	4.9	0.0	4.4	17.5	35.0
	175B	79.5	73.5	65.9	65.4	43.8	52.1	69.6	82.4	81.7	87.5
Codex	-	82.3	77.9	67.1	73.2	49.0	64.8	71.7	98.5	85.8	88.3
PaLM	8B	19.8	24.9	55.6	53.5	12.9	13.1	55.1	75.2	34.2	40.0
	62B	65.4	68.1	58.4	63.4	29.8	44.7	72.1	93.6	65.8	70.0
	540B	78.1	79.9	68.6	77.8	49.0	65.3	80.5	95.4	80.8	91.7

Figure 2.1: Comparison between Standard Prompting & CoT Prompting over five different Commonsense Reasoning benchmarks [WWS⁺23]

Wei et al. [WWS⁺23] highlight the superiority of CoT prompting strategies across diverse domains such as Commonsense Reasoning, Arithmetic Computation, and Symbolic Reasoning, among others. The paper discusses multiple datasets for each use case to exemplify the impact Chain-of-Thought has on different tasks, when compared to regular, standard prompting strategies. Figure 2.1 from [WWS⁺23] provides a comparative study between Standard Prompting and CoT Prompting over five different Commonsense Reasoning benchmarks. This comparison showcases an important finding: CoT prompting performs better for larger models with significantly more number of trainable parameters. In other words, as the model scale increases, so does the efficacy of the CoT prompting method used. Models with more than 100 billion learnable parameters show higher accuracies across tasks, and notably, smaller models often generate erroneous responses and hallucinated outputs.

The paper [WWS⁺23] also highlights certain disadvantages of the CoT prompting method. Firstly, it offers little to no improvement over the standard prompting strategies for smaller models. Infact, it is evident from Figure 2.1, that in some cases, Chain-of-Thought fares worse than standard methods. Another drawback of the method is the lack of verifiability. Of course, CoT provides a logical basis for the models to build their answers on. However, it does not equip them with ways to verify their correctness. Responses generated by the model should ideally be subjected to external verification by a human expert.

Despite its disadvantages, the strength of CoT prompting, as highlighted in [YHW⁺23], lies in its alignment with human reasoning processes. It tries to mimic the "step-by-step breaking-down of the problem" tactic that is so native to humans. A number of enhancements of traditional CoT prompting have been built around this unique property.

One limitation of baseline CoT prompting is the need for quality examples that could convince

the model to follow a set, logical thought pattern. When usage is scaled, it becomes both tedious and expensive to have humans write few-shot examples in real-time. This problem was addressed in the 2022 paper [ZZLS22] which proposed a fully automated version of CoT prompting where these black-box models generated the said few-shot examples all by themselves.

Auto-CoT completely removes the need for manually writing demonstrations for the model to learn from. First, it clusters questions of similar kinds into groups using Sentence-BERT embeddings. The k-means clustering strategy ensures that every cluster formed represents a specific "reasoning skill". This phase is referred to as Question Clustering. Then, from each cluster, a question that is most semantically similar to the cluster center would be chosen to generate a guiding example. The resulting demonstrations would then be filtered on the basis of some heuristics that uphold the quality of the required few-shot examples. Each of these remaining "filtered" examples would then serve as a few-shot example for the main question that the LLM needed to answer (called "demonstration sampling"). It is important for questions to be drawn from different clusters to enhance diversity and prevent errors from propagating through instructional training. In fact, it has been demonstrated in ablation studies that even noisy examples can be mitigated if there is enough diversity in relevant clusters.

The paper [SGS⁺23] also proposes a relatively similar solution called Synthetic Prompting. It uses zero-shot prompting on multiple questions to generate training demonstrations. These are then filtered, first on the basis of their correctness and then on the basis of diversity. The correctness filter basically uses gold labels to measure the correctness of the demonstrations (via majority voting). The diversity filter uses sentence embedding similarity to ensure that the correctness-filtered demonstrations are also varied enough to enforce generalization, resulting in top-k (usually 8) final demonstrations. These are then used as few-shot examples to reason and answer questions.

Again, Synthetic Prompting does not incur the expense of a human or an expert writing out few-shot examples for further CoT reasoning. With this method, we have a much higher capacity to incorporate more examples into our reasoning exercise for the LLM to refer to. It majorly outperforms zero-shot prompting and also performs better than few-shot CoT prompting in a number of tasks. It does, however, have its limitations. There is a need for better filtering strategies than the ones that currently exist. Filtering outputs is important as not all zero-shot demonstrations generated would be good enough. Poor examples can result in errors and inconsistencies propagating deep into the reasoning framework. Additionally, despite showcasing better results than zero-shot prompting, the method still struggles on complex reasoning tasks, thus, reiterating the need for further research in this area.

Overall, CoT improves transparency of reasoning, highlights inconsistencies, and often boosts accuracy on reasoning-heavy tasks, especially for larger models. In our preliminary experiments, we test a number of different CoT prompting strategies before agreeing on a variant to conduct detailed investigations.

2.2 Argumentative LLMs (argLLMs)

While CoT prompting deals with making the reasoning process of LLMs more explainable and transparent, the literature surrounding it does not really talk about improving the contestability of these models. At their core, LLMs suffer from what is called Sycophancy Bias, which refers to the tendency of AI to agree and please users at all costs, even if it results in a compromise in factual accuracy [Mal24]. They would not argue or engage in any kind of debate with the user over the correctness of a claim or the validity of a reasoning task. Infact, it has been observed that LLMs tend to give in when pressured or grilled repetitively over the veracity of a response, often flipping their stance and altering their "apparent" reasoning for it. This behavior is undesirable, as in critical systems this inclination to satisfy the user at the expense of correctness could result in catastrophic outcomes.

One potential solution to this issue is to employ argumentation frameworks to generate our desired demonstrations and weigh their relative strengths in the context of explainability. ArgLLMs (short for argumentative LLMs) [FDG⁺25] formalize this idea by subjecting each verifiable

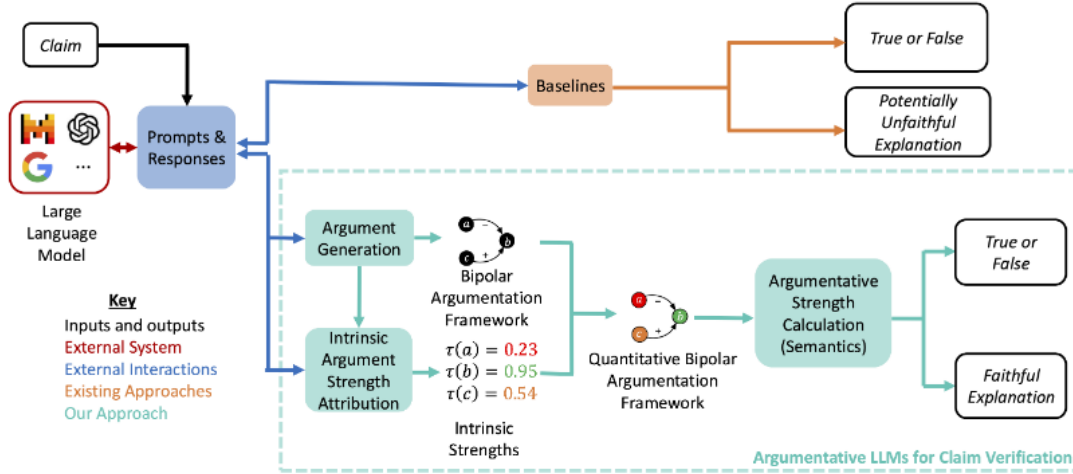


Figure 2.2: argLLM pipeline [FDG⁺25]

claim to a debate tree, classifying the arguments associated with the claim as support and attack arguments. Each claim represents the root node of the tree with a hierarchy of supporting and attacking arguments branching from it. This structure makes the reasoning process of argLLMs highly contestable and modifiable.

For a given claim, the LLM is asked to generate support and attack arguments for it. These are then organized in the form of a Bipolar Argumentation Framework (BAF), with a certain depth and breadth. Depth represents the number of layers of arguments that a claim could have. In other words, a support/attack argument could itself serve as a parent node and have supporters and attackers of its own. The breadth, on the other hand, is the number of relevant arguments a particular claim (or argument) can have (basically, how far the tree or graph can spread horizontally). The edges of the graph represent the logical relations between the nodes (claim/argument), resulting in a directed graphical structure. The paper [FDG⁺25] refers to this phase as Argument Generation. Mathematically, given a claim q , an LLM generates arguments $A = \{a_1, \dots, a_n\}$ and relations $R^+ \subseteq A \times A$ (supports) and $R^- \subseteq A \times A$ (attacks), forming the said BAF.

This is followed by the Intrinsic Argument Strength Attribution phase. Now, a base score (value between 0 and 1) is assigned to each argument. There are a couple of variants explored here: one is assigning a neutral value of 0.5 to the original claim/generated arguments; another is to allow the LLM to generate this initial value in another prompt. This gives the framework a valid and fair starting point for performing the formal evaluation. With numeric initial strengths $w : A \rightarrow [0, 1]$, we obtain a Quantitative BAF (QBAF).

We now move onto the Formal Evaluation (via Gradual Semantics) phase where the graph, now a QBAF, is passed through a formal reasoning algorithm ([ZPH⁺25], [BVPJ25]). The gradual semantics comprises an aggregation function and an influence function. Aggregation basically combines the supporting and attacking inputs to a node. On the other hand, influence propagates along the edges of the argument graph. Usual choices are the Product function for aggregation and the Linear or Euler/Runge–Kutta updates for influence. This calculates the final claim score based on all relevant supporting and attacking arguments spread over the entire tree. The final score is then thresholded over 0.5; any claim with the final score above this threshold is considered true; false if the final score is below this value. This makes the argumentation framework highly structured, far more than the reasoning generated by standard prompting or even Chain-of-Thought prompting. It also allows the user a pathway to formally edit the framework at will, consequently enabling them to alter the model’s reasoning strategies and predictions alike.

Experimental results over a number of different real-world claim verification tasks such as TruthfulClaim, StrategyClaim, and MedClaim reveal that argLLMs outshine baseline CoT implementations in terms of structural integrity, reasoning transparency, and contestability. Plus, this

high organizational prowess does not come at the cost of result accuracy (measures are still comparable to CoT prompting). Figure 2.3 from [FDG⁺25] highlights the metrics across four different variants of argLLMs based on different initial claim scores (either LLM estimated or neutral 0.5) and argument graph depths (D=1 or D=2). These insights reinforce our belief in the effectiveness of this research: that our evaluated methods for reasoning and explainability of LLM responses are viable and leave room for further enhancements.

		Direct Question	Est. Confidence	Chain-of- Thought	0.5 Base Arg (D=1)	0.5 Base Arg (D=2)	Est. Base Arg (D=1)	Est. Base Arg (D=2)
Truthful Claim	Mistral	<u>0.73</u>	<u>0.73</u>	<u>0.75</u>	0.65	0.67	0.76	<u>0.75</u>
	Mixtral	<u>0.77</u>	<u>0.77</u>	<u>0.76</u>	0.72	0.72	0.81	<u>0.78</u>
	Gemma 7B	<u>0.65</u>	<u>0.62</u>	0.68	0.64	0.62	<u>0.63</u>	<u>0.62</u>
	Gemma 2 9B	0.78	<u>0.74</u>	0.78	0.68	0.68	<u>0.73</u>	<u>0.73</u>
	Llama 3 8B	<u>0.66</u>	<u>0.69</u>	0.70	0.63	0.61	<u>0.68</u>	<u>0.69</u>
	GPT-3.5-turbo	<u>0.70</u>	<u>0.73</u>	0.74	0.60	0.64	<u>0.73</u>	<u>0.72</u>
	GPT-4o mini	<u>0.78</u>	<u>0.79</u>	<u>0.79</u>	0.74	<u>0.78</u>	0.81	0.81
Strategy Claim	Mistral	<u>0.60</u>	<u>0.61</u>	0.68	0.58	0.61	<u>0.62</u>	<u>0.60</u>
	Mixtral	<u>0.68</u>	<u>0.67</u>	<u>0.64</u>	0.62	0.66	<u>0.68</u>	0.70
	Gemma 7B	<u>0.55</u>	<u>0.56</u>	<u>0.58</u>	<u>0.56</u>	0.59	<u>0.57</u>	<u>0.57</u>
	Gemma 2 9B	<u>0.71</u>	<u>0.71</u>	0.72	0.65	0.67	<u>0.71</u>	<u>0.70</u>
	Llama 3 8B	<u>0.61</u>	<u>0.59</u>	0.65	0.54	0.53	<u>0.61</u>	<u>0.58</u>
	GPT-3.5-turbo	0.73	<u>0.70</u>	<u>0.72</u>	0.56	0.57	<u>0.70</u>	<u>0.67</u>
	GPT-4o mini	0.77	<u>0.74</u>	<u>0.75</u>	0.66	0.67	<u>0.74</u>	<u>0.75</u>
Med Claim	Mistral	<u>0.55</u>	<u>0.57</u>	0.61	0.50	0.51	<u>0.53</u>	<u>0.52</u>
	Mixtral	<u>0.60</u>	<u>0.62</u>	<u>0.61</u>	0.59	0.56	<u>0.61</u>	0.63
	Gemma 7B	<u>0.52</u>	<u>0.53</u>	0.55	0.51	<u>0.52</u>	<u>0.52</u>	<u>0.52</u>
	Gemma 2 9B	<u>0.61</u>	<u>0.58</u>	0.62	0.57	<u>0.59</u>	<u>0.60</u>	<u>0.59</u>
	Llama 3 8B	<u>0.54</u>	<u>0.56</u>	0.58	0.51	0.53	<u>0.53</u>	<u>0.52</u>
	GPT-3.5-turbo	0.67	<u>0.57</u>	0.67	0.56	0.55	<u>0.57</u>	<u>0.57</u>
	GPT-4o mini	0.74	<u>0.71</u>	<u>0.72</u>	0.62	0.65	<u>0.71</u>	<u>0.71</u>
	GPT-4o	0.85	<u>0.82</u>	0.85	0.74	0.76	<u>0.83</u>	<u>0.80</u>

Figure 2.3: Accuracy of three baselines (direct prompting, confidence estimation, CoT prompting) & four variations of ArgLLMs on claim verification tasks [FDG⁺25]

Therefore, compared to CoT, argLLMs offer greater contestability, such that users can add, delete, or modify nodes and edges, as well as observe these principled changes. They allow the evaluation of the explanation (argument graph) at the node-level, that helps identify which supports or attacks most influence the final decision.

The inherent structure of argLLMs as a framework opens it up to extensive evaluation based on various metrics that we shall discuss in the next section. These fine-grained metrics don’t just assess the quality of the individual demonstrations, but also measure how these explanations fair in the larger context of the question that the prompt seeks to address.

2.3 Explanation Properties & Evaluation Metrics

So far we have discussed the background literature exploring the existing developments in the two methods that we aim to examine and benchmark as part of our research, namely CoT prompting and argLLMs. But the question still remains: how do we make these assessments? What metrics should we use and what would each of these metrics test? Would they be sufficient to assess the impact of these demonstrations with regard to the original goal of the LLM prompt? This section discusses the relevant work done in this field and lays the foundation for further expansion on such evaluation metrics.

Firstly, in order to evaluate different explanations generated from LLMs, it is imperative to understand the different types of explanations that we can elicit from the models. The paper [KT24] categorizes LLM explanations as: free-form, deductive, and argumentative. Free-form explanations are the simplest of the lot; they are model-generated natural language summaries of the LLM’s apparent thinking process. They are the closest to human-styled explanations and are often vague & difficult to measure.

On the other hand, deductive explanations enforce a certain graphical structure to the reasoning process. [KT24] defines deductive explanations as,

"...a pair composed of a finite, non-empty sequence of propositions and a binary relation over the propositions..."

As it turns out, the explanations generated by Chain-of-Thought prompting belong to this vast family LLM explanations called deductive explanations.

Argumentative explanations are deemed to be the most structured, complex, and logic-rich form of explanations so far. As discussed in the previous section, these are primarily based on claims (or premises), arguments, and the relationships between them. They are presented in the form of debate trees with both sides being represented in the form of support and attack arguments respectively. As usual, the tree is then passed through a formal reasoning algorithm to reach the final verdict. More formally, [KT24] defines it as,

"...a 3-tuple which amounts to a finite, non-empty set of arguments, a binary attack relation over the set of arguments and a binary support relation over the set of arguments..."

Each of these types of explanations is presented with its own set of evaluation challenges. [KT24] clearly outlines a number of evaluation metrics that cater to each type of explanation discussed so far.

For free-form explanations, the idea is to assess whether the natural text explanation has a certain logical flow to it. [KT24] defines a property called Coherence $Coh(E)$ to evaluate this, where E is the explanation being evaluated. $Coh(E)$ checks if the internal subsets of a generated explanation contradict each other and if so, to what end? It does not, however, talk about the overall explanation's relation to the final conclusion.

On other hand, deductive explanations require more structurally-verifiable metrics to ensure a well-rounded, comprehensive evaluation. [KT24] introduces properties such as Non-circularity, Redundancy, and (Weak & Strong) Relevance for this purpose. Non-circularity deals with cyclicity in explanations. In other words, it evaluates whether an explanation reuses the conclusion as a justification of itself. Formally, a deductive explanation is said to be non-circular if its structure as a directed graph is acyclic. Redundancy measures the propensity of multiple intermediate reasoning steps to be very similar to each other such that they are at the risk of sounding repetitive. An explanation $(\mathcal{P}, \mathcal{R})$ is said to be non-redundant if removing any one proposition $p_i \in \mathcal{P}$, along with its associated relations, invalidates the explanation. The relevance metric is concerned about how each of the intermediate steps in the reasoning chain eventually connects to the final answer that the LLM suggests. Strong and weak connections are defined as $RelStrong(E)$ and $RelWeak(E)$ respectively.

When it comes to argumentative explanations, [KT24] defines the metrics such as Acceptability and Dialectical Non-Circularity, among others. Acceptability is defined as the fraction of arguments accepted within the semantics that support the winning label, with contradictory attacks suppressed. In other words, an argumentative explanation is acceptable when there is at least one argument in the set that cannot be defended against its attackers for low-confidence predictions, or when every argument in the set can defend themselves against all attackers for high-confidence predictions. Circularity basically measures dialectical self-support in argument graphs. An argumentative explanation is dialectically non-circular if it contains no cycles of supports or attacks, that is, no argument ends up indirectly supporting or attacking itself, thereby avoiding any contradictory reasoning structures.

These metrics provide computationally effective ways to assess the intrinsic properties of individual explanations. They are basically concerned about whether these demonstrations "make sense" internally. However, they do not really deal with the relevance of these explanations to the original goal of the reasoning exercise: the claim or question concerned. Neither do they take into account the explanation's overall alignment with the model's implicit knowledge base. Therefore, we turn our attention to another metric that attempts to address this issue, the faithfulness of explanations.

2.4 Understanding Faithfulness

Generally speaking, faithfulness asks whether a plausible-sounding explanation actually reflects the model’s true reasoning abilities and implicit knowledge base; whether the reasons provided for an answer really affect or influence the decision. At its core, faithfulness tries to establish a sound connection from plausibility to universal correctness.

Over the years, several broad evaluation paradigms for faithfulness have been explored. For example, there are counterfactual tests which alter input structures & semantics, and then record resulting prediction changes to measure faithfulness. On the other hand, input reconstruction tests have been proposed; to rebuild inputs from extracted rationales and then check the prediction consistency before and after this exercise, thereby defining faithfulness. A low-level approach to faithfulness evaluation deals with attribution, such as inspecting the internal layers and understanding the relative attention given to each part of the input claim or prompt while making the prediction.

There is ample literature available that talks about the different ways faithfulness of LLM explanations can be quantified. [MNGK25] defines faithfulness as the alignment between influential concepts & actual concepts that drive the model’s decision-making. It tries to correlate two axes: CE (causal effects) & EE (explanation-induced effects). CE measures the impact of altering a concept on the model’s response. This is done with the help of counterfactual tests. EE, on the other hand, measures how often the concept is referenced in the explanation. The final faithfulness score is the Pearson Correlation between CE and EE. In other words, if a particular explanation frequently mentions concepts that do not actually affect the outcome, it is certainly not faithful, even though it might sound plausible and convincing.

[ACL⁺23] also talks about faithfulness of Natural Language Explanations or NLEs. In addition to counterfactual tests, [ACL⁺23] also proposes an input reconstruction test to determine if the apparent reasons in the explanations match the actual reasons behind the model’s response. First, they generate an NLE from any user input and extract the key reasoning parts from them. These reasoning elements would then be used to reconstruct a new input. Predictions would be made once again and if these new predictions do not align with those made with the original input, then the explanation would not be considered sufficiently faithful. Infact, experimental results based on this test reveal that upto 40% of NLEs are not really faithful.

The paper [SCHPO24] further builds on this idea. It allows us to view counterfactual tests in a different light. Instead of simply focusing on whether an impactful intervention flips the model’s prediction, the solution also looks at the amount or degree of change in the prediction. The paper introduces two novel metrics: Correlational Explanatory Faithfulness (CEF) and Correlational Counterfactual Test (CCT). Basically, faithfulness is estimated using the Pearson Correlation between the intervention impact measure and the frequency of mention of the intervention (new tokens introduced) in the explanation. It is useful to note that the interventions used were trivial adjustments in parts of natural language speech, such as changes in adverbs and adjectives.

One of the more thought-provoking directions in faithfulness evaluation has been to analyze whether explanations (or "rationales") produced by prompting or attribution resemble human explanations and actually reflect the model’s decision-making process. The paper [FYSP24] investigates this dual challenge and shares an important perspective. The authors tested this idea for a range of open and proprietary LLMs, against datasets with annotated rationales such as e-SNLI, FEVER, and MedicalBios. They compared prompted rationales against attribution methods before and after task-specific fine-tuning. Interestingly, it was found that attribution-based rationales, especially Input×Gradient and Saliency, were consistently more human-aligned and significantly more faithful once models were fine-tuned. On the other hand, the prompted explanations neither aligned well with humans nor reliably improved with definitive fine-tuning. According to previous research, faithfulness scores have traditionally been very low across different methods of generating rationales. For example, it has been shown in [FYSP24] that masking tokens rarely changed the model’s prediction, resulting in a misleading conclusion that LLM rationales are largely unfaithful, regardless of the method used to generate them. The paper demonstrates that this was, in fact, an unreliable conclusion and a result of weak baseline models used. Once accuracy was improved

via fine-tuning, the attribution methods clearly outperformed prompting rationales in terms of faithfulness.

However, instead of literally masking out words at the semantic (or external) level, could we measure the impact of causal perturbation in the latent space? After all, a potential limitation of the method proposed in [FYSP24] is the introduction of out-of-distribution artifacts caused by unnatural masked text. Essentially, their masking-based flip-rate test forms a conceptual precursor to our proposed novel method that perturbs attention to explanation tokens to measure faithfulness, validating our emphasis on attention as the foundation of faithfulness measurement [WP19], [JW19].

To fully express the basis of our proposed evaluation test, we need to first understand the correlation between attention and explanations.

Two contradictory but highly influential papers, [JW19] and [WP19], lie at the heart of this discussion on whether attention can serve as a faithful LLM-generated explanation. Jain & Wallace [JW19] approached the problem by proposing two faithfulness criteria. Firstly, for attention to truly be an explanation of the model’s decision, then the words that get high attention should also be the words that other independent importance tests say are crucial for the prediction. These tests include input gradients and leave one out erasure, among others. Additionally, attention distributions should be unique, such that different attentions yield different prediction outcomes. These two criteria were tested rigorously across different NLP tasks such as sentiment analysis, news classification, medical coding, NLI, and QA, and it was revealed that neither of the conditions are consistently met. Correlations between attention and gradients/erasure tests were often very weak and they also led to randomly permuted attention distributions that differed drastically from originals, even though predictions were almost always unchanged. This subverted the notion that attention heatmaps are causally faithful explanations. However, Wiegrefe & Pinter [WP19] strongly responded to this paper and argued that this critique was too extreme and that the counterfactual tests conducted were not particularly realistic. Basically, they contended that the adversarial attention weights created by Jain & Wallace were not true to the model and, naturally, could not represent how the model internally works. They proposed a number of structured tests to support their arguments. For example, they trained consistent adversarial models that jointly preserved predictions while diverging attentions. Results showed that in tasks where attention improves performance, base attentions transfer useful token-importance signals, while adversarial attentions do not. This effectively dismissed the efficacy of adversarial attentions as a valid counterfactual test, at least in such cases. [WP19] concluded that while attention is not always faithful, it still serves as a useful form of plausible explanation, particularly when evaluated in a way that respects the model’s architecture.

Overall, [JW19] highlights the errors in considering raw attention scores as faithful explanations, motivating our shift from simply interpreting attention distributions to perturbing them and measuring causal prediction changes. [WP19], on the other hand, emphasizes on the need to validate such counterfactual tests responsibly, leading us to run extensive model-consistent experiments across datasets to cover different use cases.

In another paper, Liu et al., 2022 [LLG⁺22] inspires us to view LLM faithfulness through the eyes of attention weights and causal influence. They introduce a Faithfulness Evaluation Test (FVT) that entails perturbing the most highly weighted feature (for example: tokens, in case of textual models) either via deletion, masking, or replacement. The central idea of the test is that most attended token has the highest influence on the model’s decision making. Therefore, any perturbation on it should lead to a decrease in the prediction confidence. Therefore, the model’s confidence is then checked for any decrease or increase given this perturbation. It was found that for numerous datasets, architectures, and explanation methods, this criterion was severely violated. Instead of an expected decrease in said confidence, there an observed increase that made the explanations misleading, especially for bigger, deeper models like transformers. As we propose our novel attention perturbation test through this research, we take FVT’s input masking a step further. Instead of altering sentence semantics that could lead to out-of-distribution (OOD) inputs, our new method perturbs the model’s own attention modules to the extracted rationale tokens, providing a model-aware counterfactual test of faithfulness, while preserving the structure of the original input prompt (or claim). The usage of transformers-based LLMs further reinforces the need for such a comprehensive and multidimensional test.

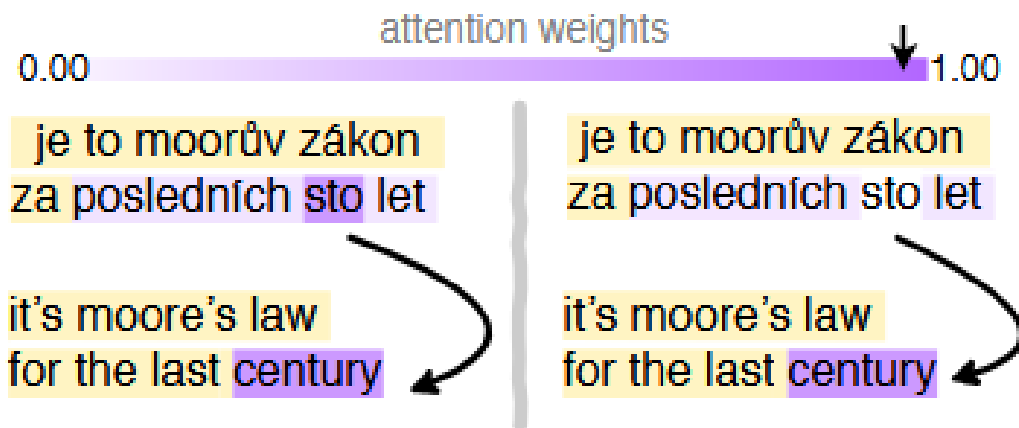


Figure 2.4: This example from a Czech-to-English translation highlights how a model’s attention mechanism can be unfaithful; (a) [to the left] when translating the word *sto* into *century*, an attention heatmap shows the model is correctly focusing on the word *sto*; (b) [to the right] however, it’s also possible for the model to generate the exact same word, *century*, even when the heatmap shows it is not paying any attention to *sto* [MKS21]

In 2021, Moradi et al. [MKS21] proposed a Stress Test, particularly for Neural Machine Translation (NMT) tasks, that evaluated whether attention weights actually function as faithful explanations for model predictions. This alignment between visual plausibility and causal faithfulness was explored in a familiar fashion: perturbing attention distributions. Several different methods were tested, such as ZeroOutMax (zeroing out attention to most attended token), Uniform (replacing original weight distribution with equal weights), and random combinations, and subsequently, their apparent impact on the translation quality was observed. It was found that translation quality remained largely unaffected even when highly attended tokens were perturbed. There were cases where the random attention weights performed as well as original ones, indicating that attention weights were not majorly or uniquely responsible for model’s outcomes. This paper plays key role in inspiring the design of our SHAKE test, as it demonstrates that attention or attribution alone is not a good enough measure of faithfulness, just like counterfactuals alone aren’t either. It is important to come up with a method that combines the best of both worlds and illustrates the separation between plausibility and faithfulness. Moreover, the Stress Test from [MKS21] is training-integrated and modifies the attention distribution itself, while our novel faithfulness evaluation test perturbs the attention weights to the explanation tokens only, in a post-hoc manner. This makes our method suitable for deep, multi-head, multi-layer transformers-based LLMs, with highly distributed attention, as opposed to the relatively simpler seq2seq models on which the above stress tests were built.

Thus, our background research highlights an important notion: a fundamentally "good" explanation not only makes sense independently; it also causally matches what triggered the model’s behavior. This calls for further research in the area: a benchmark of how different models perform using CoT & argLLMs, and a novel faithfulness metric/evaluation test that examines whether an explanation is indeed valuable and effective in highlighting the true reasoning abilities of the large language model.

Chapter 3

Theoretical Foundations

This chapter elaborates on the theory behind the fundamental building blocks of our LLM explanation evaluation benchmark, as well as the proposed novel faithfulness evaluation test. We will discuss the formal definitions, the underlying math, and representative examples of the properties and metrics used to compile our evaluation suite. We will also take a closer look at the theoretical basis for the attention perturbation test, its relevance to our employed explanation paradigms, and the potential ways to measure the outcome. It is imperative to highlight these notions before we dive deeper into the implementation strategies.

3.1 Metric Definitions and Theory

3.1.1 Deductive Metrics

For deductive explanations (that is, via Chain-of-Thought prompting), we evaluate three intrinsic properties that demonstrate the internal structure and veracity of the demonstration. These are Weak Relevance, Strong Relevance, and Redundancy. Each of these highlights a distinct aspect of the said explanation and is crucial for understanding the overall quality, usefulness, and effectiveness of the reasoning chain.

Weak Relevance

A deductive explanation $\mathcal{E} : \langle \mathcal{P}, \mathcal{I} \rangle$, with propositions $\mathcal{P}$ and inference relations $\mathcal{I}$, is said to be *weakly relevant* if every proposition is connected by some inference path to the final conclusion $\hat{y}$. More formally, for each $p_i \in \mathcal{P}$, there must exist a path

$$(p_i, p_{i_1}), (p_{i_1}, p_{i_2}), \dots, (p_{i_k}, \hat{y}) \in \mathcal{I}.$$

This ensures that no intermediate step in the entire reasoning chain is disconnected from the final claim. The metric is quantified as:

$$\text{Rel}_{\text{weak}}(\mathcal{E}) = \frac{1}{|\mathcal{P}|} \sum_{p_i \in \mathcal{P}} \text{path}(p_i, \hat{y}),$$

where $\text{path}(p_i, \hat{y})$ is a binary function such that,

$$\text{path}(p_i, \hat{y}) = \begin{cases} 1 & \text{if a directed path exists in } \mathcal{I} \text{ between proposition } p_i \text{ and } \hat{y}, \\ 0 & \text{otherwise.} \end{cases}$$

Example: Suppose an explanation graph has edges $A_1 \rightarrow A_2 \rightarrow A_3$, with prediction $\hat{y} = A_3$. All nodes can reach A_3 , so $\text{Rel}_{\text{weak}} = 1.0$. If a node A_4 exists with no path to A_3 , then only 3/4 nodes are connected, giving $\text{Rel}_{\text{weak}} = 0.75$. In practice, whether a particular proposition p_i can reach another proposition p_j in the reasoning chain is determined by a method called natural language inference (NLI) and further discussed in the next chapter of this report.

Strong Relevance

An explanation is considered *strongly relevant* if every proposition is directly connected to the conclusion $\hat{y}$. In other words, for all $p_i \in \mathcal{P}$, the edge $(p_i, \hat{y})$ exists in $\mathcal{I}$. This is a much stricter condition than weak relevance, as it requires a direct connection between a proposition and the conclusion, as opposed to any indirect connections.

The metric is defined as:

$$\text{Rel}_{\text{strong}}(\mathcal{E}) = \frac{1}{|\mathcal{P}|} \sum_{p_i \in \mathcal{P}} \text{is_relation}(p_i, \hat{y}),$$

where $\text{is_relation}(p_i, \hat{y})$ is also a binary function such that,

$$\text{is_relation}(p_i, \hat{y}) = \begin{cases} 1 & \text{if } (p_i, \hat{y}) \in \mathcal{I}, \\ 0 & \text{otherwise.} \end{cases}$$

Example: If each proposition in the explanation directly points to the conclusion, then $\text{Rel}_{\text{strong}} = 1.0$. If only 2 of 5 propositions directly connect to $\hat{y}$, then the score is 0.4. It is extremely unlikely that a true CoT demonstration will have a very high strong relevance score near to 1.

Redundancy

A deductive explanation is said to be *non-redundant* if no proposition can be removed without weakening or breaking the justification for the conclusion. Basically, redundancy measures the extent to which an explanation contains reasoning that is disconnected from the central chain.

The metric is computed as:

$$\text{Red}(\mathcal{E}) = 1 - \frac{1}{|\mathcal{P}|} \sum_{p_i \in \mathcal{P}} \mathbb{K}[p_i \neq \hat{y} \wedge p_i \in \text{component}(\mathcal{I})],$$

where the redundancy score is 0 for a perfectly non-redundant explanation and 1 for a highly redundant explanation. Additionally, if the explanation can be split into multiple disconnected components, the redundancy is set to 1 (maximum; highly redundant).

Example: Consider the explanation graph $A \rightarrow B \rightarrow \hat{y}$, and $C \rightarrow A$. Every node contributes to a unique chain to $\hat{y}$, so redundancy is low. But if we have an explanation that can be split into two graphs $A \rightarrow B$ and $C \rightarrow \hat{y}$, then its redundancy is considered to be the highest at 1.

3.1.2 Argumentative Metrics

For argumentative explanations, we focus on two key properties derived from formal argumentation theory: acceptability and non-circularity. These distinct properties identify whether an argLLM explanation is both logically defensible and free of cycles of self-support or self-attack. In essence, these metrics describe complementary aspects of the quality of argLLM explanations.

Acceptability

An argumentative explanation $\mathcal{E}$ is said to be *acceptable* if the final conclusion $\hat{y}$ can be defended against all of the attacking arguments. More formally, given an argumentative explanation

$$\mathcal{E} = \langle A, R_{\text{sup}}, R_{\text{att}} \rangle,$$

where A is the set of arguments, R_{sup} the support relations, and R_{att} the attack relations, then $\mathcal{E}$ is acceptable if, whenever some $a_j \in A$ attacks an argument a_i supporting $\hat{y}$, there exists another argument $a_k \in A$ that counter-attacks a_j . In other words, every attacker must itself be attacked by some "defender" to ensure that the argument *for* $\hat{y}$ is always defended.

The acceptability metric defined in [KT24] captures this as:

$$\text{Acc}(\mathcal{E}) = \frac{1}{|A|} \sum_{a_i \in A} \left(\frac{1}{|\text{Atts}(a_i)|} \sum_{(a_j, a_i) \in R_{\text{att}}} \delta(a_j) \right),$$

where $\text{Atts}(a_i)$ denotes the set of attackers of a_i , and $\delta(a_j) = 1$ if a_j is itself has an attacker and is not undisputed.

Intuitively, $\text{Acc}(\mathcal{E})$ for an argLLM explanation should be high (close to 1) if the prediction confidence for $\hat{y}$ is high. Inversely, if $\hat{y}$ is left undefended, acceptability is low.

Example: Suppose arguments A_1, A_2 are supporting $\hat{y}$, with attackers B_1 and B_2 attacking A_1 and A_2 , respectively. Assuming both B_1 and B_2 are in turn, counter-attacked by C_1 and C_2 respectively, then $\text{Acc}(\mathcal{E}) = 1$. However, in case B_2 is left undefeated, then $\text{Acc}(\mathcal{E}) < 1$, indicating that not all attacks are countered.

Circularity

Another property, *dialectical non-circularity*, requires that no argument in an argumentative explanation supports or attacks itself, either directly or through extended cycles of reasoning. A violation of this property would result in a circular reasoning scenario, where an argument’s justification depends on itself, diminishing the quality of the explanation.

Again, formally, an explanation $\mathcal{E} = \langle A, R_{\text{sup}}, R_{\text{att}} \rangle$ is non-circular if there exists no sequence of arguments

$$A' = \{a_1, a_2, \dots, a_k\} \subseteq A$$

such that $(a_i, a_{i+1}) \in (R_{\text{sup}} \cup R_{\text{att}})$ for all $1 \leq i < k$, and $a_1 = a_k$. In other words, there must be no cycle in the graph of support and attack relations.

The circularity metric is thus defined as:

$$\text{Circ}(\mathcal{E}) = \frac{1}{|A|} \sum_{a \in A} \frac{1}{M} \sum_{A' \subseteq (R_{\text{sup}} \cup R_{\text{att}})} \mathbf{1}[\text{cycle}(A')],$$

where $\mathbf{1}[\text{cycle}(A')] = 1$ if a cycle is detected among arguments in A' . Lower values of $\text{Circ}(\mathcal{E})$ indicate presence of better argLLM explanations.

Example: Suppose we have arguments (supporters or attackers) A_1, A_2, A_3 with relations $A_1 \rightarrow A_2$, $A_2 \rightarrow A_3$, and $A_3 \rightarrow A_1$. This forms a structural cycle among the arguments, so $\text{Circ}(\mathcal{E}) > 0$. On the other hand, if $A_1 \rightarrow A_2$ and $A_2 \rightarrow \hat{y}$ with no back-links to A_1 from A_2 then $\text{Circ}(\mathcal{E}) = 0$, indicating that no circular reasoning exists.

However, the above definitions are purely structural. This highlights a key limitation in their usage, that is, their failure to consider what the arguments actually say. They capture those properties of the graph $\langle A, R_{\text{sup}}, R_{\text{att}} \rangle$ that make it a structurally "good" explanation. This leads to a couple of observations. Firstly, at depth $D = 1$, acceptability will never have a non-zero value considering every claim has an attacker, but this attacker does not have any other attacking argument at the next level. Circularity too, will never be observed at any depth D considering the structurally different definitions of the arguments. Thus, simply structural considerations of the above properties (or metrics) would lead to trivial values and convey little about the true quality of the argLLM explanation.

Therefore, to comprehensively understand said explanations in terms of acceptability and circularity, we extend our interpretation of these metrics to capture semantic similarity between the support/attack arguments and the original claim. Let $\phi(\cdot)$ be a sentence embedding function and let $\text{sim}(x, y) \in [0, 1]$ denote cosine similarity between $\phi(x)$ and $\phi(y)$. Let $S(\hat{y}) = \{s : (s, \hat{y}) \in R_{\text{sup}}\}$ and $A(\hat{y}) = \{a : (a, \hat{y}) \in R_{\text{att}}\}$. We introduce two threshold hyperparameters τ_{circ} and τ_{neu} .

Semantic Circularity: The idea behind this interpretation of circularity is that supporters should contribute *new* reasons for $\hat{y}$, not rephrase it. It is possible that, while generating the explanation that comprises an entire bag of support and attack arguments, the supporters might be similar to the original claim, so much so that they are basically paraphrases. Referring to the definition of circularity above, in such a scenario, the head and the tail of the relation would be identical, that is, the claim itself. Therefore, if a supporter is too similar to the claim, there is circularity in the argLLM explanation:

$$\text{SCirc}(\mathcal{E}) = \begin{cases} \frac{1}{|S(\hat{y})|} \sum_{s \in S(\hat{y})} \mathbf{1}[\text{sim}(\phi(s), \phi(\hat{y})) \geq \tau_{\text{circ}}], & |S(\hat{y})| > 0, \\ 0, & |S(\hat{y})| = 0 \end{cases}$$

where τ_{circ} is the threshold value for circularity.

Semantic Acceptability: Intuitively, attackers should genuinely oppose the claim and its supporters. So conversely, if an attacker a is semantically close either to the claim or to a supporter of the claim, then it is considered to be non-contradictory and is treated as neutralized. More formally,

$$\text{SAcc}(\mathcal{E}) = \begin{cases} 1, & |A(\hat{y})| = 0, \\ \frac{1}{|A(\hat{y})|} \sum_{a \in A(\hat{y})} \mathbf{1}\left[\max\left(\text{sim}(\phi(a), \phi(\hat{y})), \max_{s \in S(\hat{y})} \text{sim}(\phi(a), \phi(s))\right) \geq \tau_{\text{neu}}\right], & |A(\hat{y})| > 0 \end{cases}$$

where τ_{neu} is the threshold value for acceptability. Here a higher SAcc means that a larger fraction of attackers are semantically redundant to the claim or its supporters, and hence, do not undermine $\hat{y}$.

Logically, a truly robust argLLM explanation would usually balance both properties, acceptability and circularity, defending itself against attacks, while avoiding reasoning loops.

3.2 Faithfulness: Attention Perturbation Test

In XAI, the concept of faithfulness is a critical property that describes both, the model’s reasoning prowess and its ability to reproduce this reasoning in its responses. The term has been extensively used to verify whether a plausible-sounding explanation is truly rooted in the model’s computational logic or is merely a post-hoc rationalization, or worse, a hallucination. We have already discussed the different interpretations of faithfulness that have been explored over the years. Now, let us look at the theory behind our novel faithfulness evaluation test and understand the architecture that supports and propels it.

The evaluation metrics discussed in section 3.1 describe, examine, and quantify the internal quality of an explanation. However, faithfulness cannot be ascertained via these static structural metrics as these only capture necessary conditions for intrinsically "good" explanations. To evaluate whether model-provided rationales are truly reflective of its final decision, we must probe directly into the model’s internal architecture. This motivates our novel attention perturbation test: **S**elf-attention **H**euristic for **A**tributational **K**eyword **E**limination, or the **SHAKE** test.

3.2.1 Conceptual Basis

Let $\mathcal{M}$ be a transformer-based large language model that outputs a binary label $y \in \{\text{TRUE}, \text{FALSE}\}$ when prompted with an input claim $\mathcal{X}$. Additionally, along with the label, the model was asked to print an explanation in the form of a set of "keywords" or explanation (or rationale) tokens $R = \{t_{i_1}, \dots, t_{i_r}\}$; where $R \subseteq \mathcal{X}$, that it believes causally influenced its final decision, i.e., the prediction label. Faithfulness requires that these word-level rationale tokens, that would eventually make up the natural language explanation, truly influence the model’s prediction. To test this, the SHAKE test perturbs the model’s internal attention mechanism such that it can no longer attend

to R . If the prediction label subsequently changes after the perturbation exercise, then the explanation is deemed faithful, because then it would indicate that the declared rationale was indeed important for the model’s decision-making. However, if the label does not change, the explanation is considered unfaithful.

3.2.2 Rationale Token Selection

The SHAKE pipeline identifies the rationale token set R using a lightweight keyword extraction step. Unlike the usual attributional methods that derive important tokens through gradients or attention weights, SHAKE directly and explicitly asks the LLM for a concise set of rationales. This ensures that the following perturbation test is true to the model’s own stated reasoning, thereby allowing direct faithfulness evaluation. For each input claim $\mathcal{X}$, the model is prompted to print upto top- K word-level rationale tokens considered pivotal to its final decision. This constraint K on the number of explanation tokens to print prevents large, overused rationales (such as entire sentences) from being attention-perturbed and eventually compels the model to stick to concise explanations that are most relevant to the model’s internal reasoning.

$$R = \{t_{i_1}, t_{i_2}, \dots, t_{i_r}\}, \quad \text{with } r \leq K.$$

The value of K was assigned as 4 for the purpose of these experiments. Extensive probing into the claim verification datasets, particularly *TruthfulClaim* and *StrategyClaim*, revealed a crucial piece of information. Almost all claims basically talked about 2-3 key concepts in their statement, such that the main intuition behind these could easily be covered using a maximum of 4 keywords. Limiting the value of K to ≤ 4 also prevented over-perturbation of statements, resulting in faulty prediction outcomes which would have no real meaning. However, in future experiments, we could certainly play around with the value of K to see how these different perturbation variants perform.

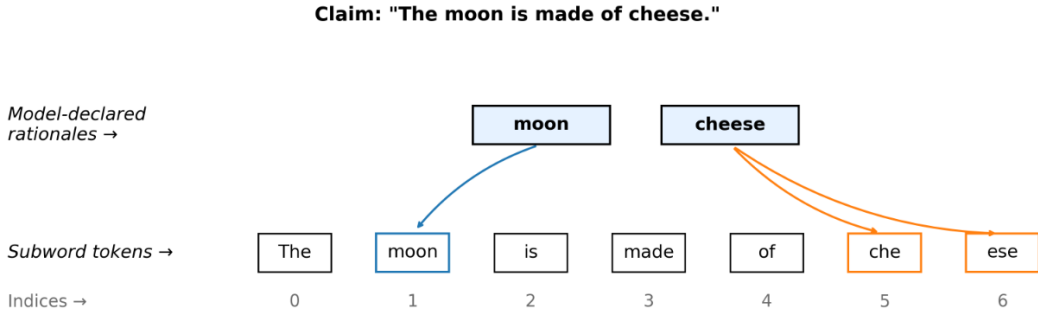


Figure 3.1: Example rationale token selection and alignment; model-declared rationale words aligned to sub-word indices; (a) 'moon' $\rightarrow$ tokens[1] ('moon'); (b) 'cheese' $\rightarrow$ tokens[5:7] ('che', 'ese')

Once these keywords are returned, they are matched against the original claim $\mathcal{X}$ at the tokenization level. The tokenizer’s character-to-token alignment is used to identify the exact subword indices corresponding to each keyword in R . This mapping is important as LLMs internally operate on subword token units (for example, Byte-Pair Encoding or SentencePiece), while the model’s rationale output is at the word level [LL25], [VLP⁺25]). Without this precise character-to-token alignment, perturbations could erroneously pick subword fragments that are not a part of the highlighted explanation, thereby invalidating the authenticity of our test. Therefore, matching the rationale set directly to valid token indices ensures that the perturbations affect only those specific tokens the model itself highlighted as valid rationales.

In other words, this method of selecting particular rationale tokens ensures that the subsequent perturbation test targets a semantically valid and interpretable set of tokens rather than an arbitrary subset of the input. It eliminates the cost of human experts who would otherwise be needed for manual rationale selection, based on hours of potentially bias-inducing exploration.

The token-level mapping method also ensures robustness across different tokenization procedures, making our SHAKE pipeline readily adaptable to different transformer architectures.

3.2.3 Attention Perturbation Mechanism

Let $A \in \mathbb{R}^{H \times T \times T}$ denote the attention probabilities at a transformer layer, where H is the number of heads and T the input sequence length. For each query position q , the attention distribution over key tokens is:

$$\alpha_{qk} = \frac{\exp(A_{qk})}{\sum_{j=1}^T \exp(A_{qj})}.$$

In SHAKE, all columns corresponding to rationale tokens R are zeroed out:

$$\alpha'_{qk} = \begin{cases} 0, & k \in R, \\ \frac{\alpha_{qk}}{\sum_{j \notin R} \alpha_{qj}}, & k \notin R. \end{cases}$$

This essentially prevents the model from "looking at" its own highlighted rationale tokens, while renormalising the remaining attention distribution. In practice, the perturbation is applied to only the last self-attention layer of the transformer-based model. This is done by replacing the default attention module with a modified custom attention module that assigns zeroed attention weights to those particular tokens or words, preserving the rest of the attention architecture of the model to work naturally.

Why is only the last self-attention layer perturbed?

In transformer-based models, rationale importance evolves as we pass over different layers. Early layers focus on surface-level pattern recognition and diffuse dependencies, while later layers interpret task-relevant semantics and drive the logits. In other words, the later layers are most aligned with the model's final decision for a given task. In our case, the tokens that the model considers to be part of the rationale also evolve as we move from the initial to the later layers. Perturbing all layers would add excessive noise to the model that would propagate through the layers and destabilize predictions. But targeting the last layers directly means that we are perturbing tokens most likely to be the actual explanation tokens for the model, thereby testing whether the extracted tokens truly matter for the model's output. This makes the SHAKE test more faithful and interpretable, as it intervenes at the level of representation the model actually used to generate its answer. This is experimentally verified and we will discuss this further in the chapter.

3.2.4 Faithfulness Evaluation via Label Flips

Let y be the original label and $\tilde{y}$ the label predicted after perturbation. The SHAKE test defines faithfulness in binary terms:

$$\text{Faithful}(E) = \begin{cases} 1, & y \neq \tilde{y}, \\ 0, & y = \tilde{y}. \end{cases}$$

Intuitively, if the model's decision truly depends on the tokens it highlighted, then preventing the model from looking at those tokens when making a decision should have a causal effect on the outcome, resulting in a label flip. Thus, if removing attention to rationale tokens causes a label flip from TRUE to FALSE (or vice versa), then the explanation is considered faithful, as this indicates that the model actually relied on those tokens in question. Conversely, if the prediction label remains unchanged, the explanation is considered unfaithful, as the said rationale did not truly contribute to the final decision.

Chapter 4

Implementation

4.1 Overview

The implementation of this project spans three distinct yet interconnected pipelines:

- Generation of Chain-of-Thought (CoT) prompted explanations and their evaluation using a set of established deductive metrics from [KT24]
- Generation of argLLM-based explanations and their evaluation using a set of established argumentative metrics from [KT24]
- Formaluation of the novel SHAKE test pipeline, evaluating faithfulness of LLM explanations through final-layer attention perturbation tests

Each of these pipelines is integrated into the implementation keeping in mind code modularity and reusability, allowing different functionalities, tests, and analyses to be added in a smooth and phased manner. The pipelines are designed to be dataset and model agnostic so that they can be applied to more and more datasets and models as per need. Particularly in case of the SHAKE pipeline, while the specific hooks and perturbations may differ between model architectures (example, LLaMA, or Mistral), the core methodology that drives the evaluation tests has been defined in a general way that enables extension to different LLM variants.

As an overview, the workflow can be seen as a clear dataflow pipeline. Data from renowned claim verification (and question answering) datasets are passed through explanation generators, and the resulting explanations are evaluated using established paradigm-specific metrics. This high-level process is illustrated in Figure 4.1, which depicts the flow of data from raw claims and questions through explanation generators, subjected to extensive evaluation, and then finally compiled into analytical reports. In this section, we also detail the design of our pilot user study that drives the comparison between computation evaluation of properties vs. human perceived usefulness of explanations.

4.1.1 Datasets Used

Let us now look at the different datasets we used to perform our experiments. We focused our research on two main use cases central to the LLM response and explanation generation: Question Answering and Claim Verification. Therefore, we employed renowned QA datasets in their native form that represented different reasoning paradigms. For claim verification, the QA dataset instances were systematically converted into binary claims. Thus, the models were evaluated at large for both, open-ended generation and discrete outputs.

Question Answering

TruthfulQA: Designed to check whether LLMs are susceptible to popular misconceptions or common myths; contains questions across diverse domains such as health, history, science, and universal truths, where naive models tend to generate untruthful but widely repeated statements.

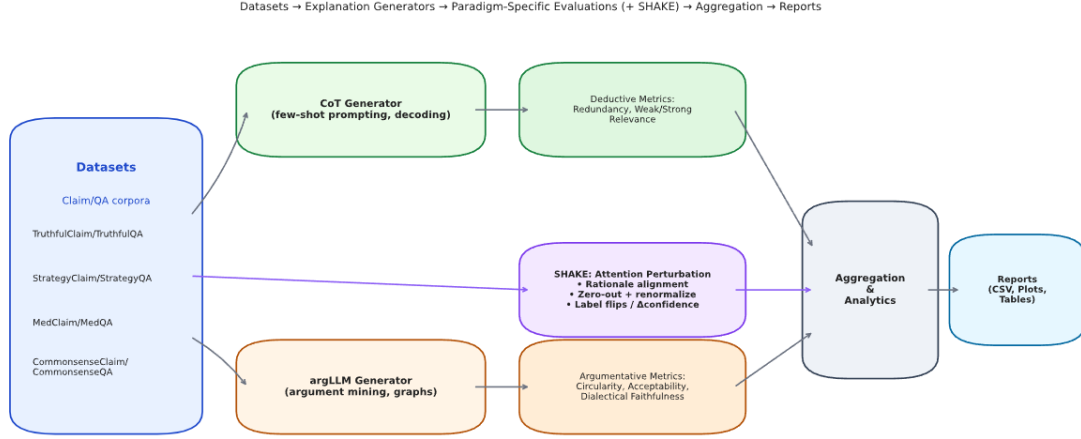


Figure 4.1: High-level overview of the project implementation pipeline, from datasets to explanation generation, evaluation, and reporting.

Listing 4.1: Dataset structure of TruthfulQA (validation split)

```
DatasetDict({
  validation: Dataset({
    features: [
      'type',
      'category',
      'question',
      'best_answer',
      'correct_answers',
      'incorrect_answers',
      'source'
    ],
    num_rows: 817
  })
})
```

Each row comprises of a **question**, a **best_answer** provided and verified by human experts, a list of **correct_answers**, and a set of **incorrect_answers** corresponding to common misconceptions.

MedQA: Consists of medical multiple-choice questions drawn from professional medical licensing examinations (for example, USMLE). It is not a general-domain dataset, but instead, requires specialized biomedical reasoning abilities to be interpreted. The reasoning generated for samples from this dataset are usually expected to be long and verbose. Questions follow a case-like pattern, where complex medical scenarios are presented that need to be deeply examined to reach the correct answer. The dataset structure is as follows: -

Listing 4.2: Dataset structure of MedQA (test split)

```
DatasetDict({
  test: Dataset({
    features: [
      'question',
      'answer',
    ]
  })
})
```

```

        'options',
        'meta_info',
        'answer_idx',
        'metamap_phrases'
    ],
    num_rows: 1273
})
})

```

StrategyQA: Here, the questions require extensive multi-step reasoning in order to be answered correctly; they lack enough information for direct and trivial answers, and require the solver to piece together intermediate facts and knowledge. It is expected that this dataset would thoroughly test the ability of the model to probe and extract the hidden reasoning steps that help reach a plausible and correct answer.

Listing 4.3: Dataset structure of StrategyQA (test split)

```

DatasetDict({
  test: Dataset({
    features: [
      'qid',
      'term',
      'description',
      'question',
      'answer',
      'facts',
      'decomposition'
    ],
    num_rows: 2290
  })
})

```

CommonsenseQA: Evaluates commonsense reasoning (as the name suggests) using questions based on regular, everyday knowledge of surroundings. For explanation evaluation, it highlights whether models can ground their reasoning in common but essential real-world knowledge. The dataset structure is as below,

Listing 4.4: Dataset structure of CommonSenseQA (validation split)

```

DatasetDict({
  validation: Dataset({
    features: [
      'id',
      'question',
      'question_concept',
      'choices',
      'answerKey'
    ],
    num_rows: 1221
  })
})

```

Claim Verification

To comprehensively evaluate explanations from CoT prompting and argLLMs, we consider another popular use case: binary claim verification. The datasets for this use case were directly derived from the QA dataset corpora. Basically, every QA data sample (from all the variants) was pass through an LLM to convert from the question format to the claim format, with labels indicating whether the claim is true or false, on a random basis. For example, a QA prompt such as “*What happens if an earthworm is cut in half?*” is converted into the corresponding factual claim “*An earthworm survives and regenerates when cut in half.*”, with the associated ground truth. The structure of these claim verification datasets generated also allowed a more grounded evaluation of

faithfulness using our novel perturbation test, as the binary labels made it easier to measure label flips across models. The dataset schemas are given below.

TruthfulClaim: Converted from TruthfulQA, each data sample comprises:

- **claim:** declarative claim statement (to be verified)
- **label:** ground truth label (TRUE or FALSE)
- **question:** original QA question from which the claim was derived
- **source:** the reference that supports the claim and corresponding claim
- **id:** unique identifier for the data sample

MedClaim: Converted from MedQA, each MedClaim sample consists of:

- **id:** unique identifier for the data sample
- **question:** original medical case/scenario from which the claim was derived
- **option:** correct multiple-choice answer to the original question
- **claim:** declarative claim statement (to be verified), derived from question and corresponding options
- **label:** ground truth label (TRUE or FALSE)
- **source_idx:** index linking back to the original MedQA data sample

StrategyClaim: Converted from StrategyQA, StrategyClaim comprises: -

- **qid:** unique identifier for the data sample
- **question:** original strategic question from which the claim was derived
- **claim:** declarative claim statement (to be verified), derived from question
- **label/answer:** ground truth label (TRUE or FALSE)

CommonsenseClaim: Converted from CommonSenseQA, CommonsenseClaim has the following structure: -

- **id:** unique identifier for the data sample
- **question:** original commonsense multiple-choice question from which the claim was derived
- **option:** the answer option chosen from the multiple-choice set.
- **claim:** declarative claim statement (to be verified)
- **label:** ground truth label (TRUE or FALSE)
- **source_id:** hashed identifier linking back to original CommonSenseQA data sample

4.1.2 Models Used

We performed our experiments on several generative models for explanation generation using CoT & argLLMs, running NLI for CoT metric computation & evaluation, and also structured rationale generation for our SHAKE pipeline.

For Explanation Generation: Decoder-only models from three renowned LLM families were used for generating the CoT and argLLM under evaluation:

- `mistralai/Mistral-7B-Instruct-v0.1`: a 7-billion parameter open-weight model, optimized for instruction following and reasoning tasks
- `Qwen/Qwen3-8B`: an 8-billion parameter model from the Qwen family, chosen for its strong reasoning abilities and long explanations
- `meta-llama/Meta-Llama-3-8B`: a third-generation 8-billion parameter model from Meta, considered to be efficient for a number of tasks

For CoT and argLLM evaluation: For constructing the adjacency matrix of propositions required for evaluating CoT explanations against the defined relevance and redundancy metrics, we employed a fine-tuned BERT classifier trained on the Multi-Genre Natural Language Inference (MNLI) dataset. Namely, the `bert-base-uncased-MNLI` model, is used for its lightweight nature and ability to detect entailment and contradiction in natural language textual pairs. In case of argLLM evaluation, we use the SentenceTransformer model, `sentence-transformers/all-mpnet-base-v2` to embed all arguments (including claim statements) and subsequently, estimate their cosine similarity to measure the semantic proximity between each other. Once again, the lightweight nature of the model justifies this choice over other larger ones.

For the SHAKE Pipeline: The attention perturbation test framework required models that made their internal attention layers visible and available for modification. This allowed us to implement custom attention hooks and replace the last-layer self-attention module of these models: (a) `Qwen/Qwen2.5-7B-Instruct` (b) `mistralai/Mistral-7B-Instruct-v0.1` (c) `meta-llama/Llama-2-7b-chat-hf`

4.2 Chain-of-Thought (CoT) Pipeline

This section details the implementation strategies adopted to build the CoT pipeline of this study. We discuss how CoT prompts that elicit explanations are constructed, how explanations are generated, stored, and parsed into constituent steps or propositions, how the adjacency matrix for the reasoning chain is built, and finally, how the defined deductive metrics are computed and eventually summarized. We also describe certain performance optimization methods employed, such as intelligent sampling of relevant propositions from verbose explanations.

The pipeline can be broken down into four sub-tasks: (1) generating step-by-step explanations, (2) representing explanations as reasoning chains through NLI, (3) evaluating said reasoning chains via given dedicated metrics, and finally, (4) aggregating evaluation results to obtain per-dataset, per-model analysis. We will now look at each of the aforementioned tasks in detail, with accompanying algorithmic representations that highlight their underlying principles of operation.

4.2.1 Explanation Generation

The first step in this pipeline was to obtain explanations for inputs from these decoder-only LLMs using the CoT prompting strategy. Since we explored both, question answering and claim verification tasks in our experiments, an input was either a claim or a question. Data from both QA and claim verification datasets (discussed in the previous section) were sampled, and passed through the LLMs. In order to generate an explanation along with the final answer, it was necessary to prompt the model accordingly. A carefully designed prompt template was used that not only asked the model to predict an answer but also explicitly requested intermediate reasoning steps that eventually led the model to such an answer. Consequently, the model’s response was not limited to a binary label or even a short, non-descriptive answer. Instead, the output was elaborate, structured, and in a step-wise manner, that allowed us to represent it as a sequence of individual propositions.

Algorithm 1: CoT Explanation Generation (Claim Verification)

Input: JSON file of claims with fields: `id`, `claim`, `label`, `question`
Output: JSONL file with `id`, `claim`, `label`, `question`, and `cot_explanation`

```
procedure Generate_CoT_Explanations(input_file, model, output_file):  
    dataset ← LoadJSON(input_file)  
    CreateDirectoryIfNotExists(output_file)  
    Open output_file for writing  
    for each entry in dataset do  
        claim ← entry.claim  
        label ← entry.label  
        id ← entry.id  
        question ← entry.question  
        prompt ← BuildPrompt(claim)                                // embed claim into CoT template  
  
        response ← QueryModel(model, prompt)  
        if response is valid then  
            | explanation ← response.text  
        else  
            | explanation ← "[ERROR] response missing"  
        record ← { id, claim, label, question, cot_explanation: explanation }  
        WriteLine(output_file, record)  
    Close(output_file)  
    return output_file  
end procedure
```

Algorithm 1 outlines the end-to-end process of generating a CoT explanations for claim verification tasks. We begin by loading the required claim verification dataset in JSON format and defining an output file for storing the generated explanations. For each claim, a CoT prompt with a stipulated structure is used that encourages step-by-step reasoning and compels the model to "think out aloud." The prompt also specifies that the model's response should have a final concluding verdict. The prompt is then sent to the particular LLM in question, whose response is then validated and parsed into an explanation. Each explanation, along with the data sample's original metadata comprising of (claim, label, question, id), is printed line-by-line to a JSONL file. This file format stores the explanations in such a way that makes it suitable for the subsequent evaluation.

Prompting strategy: For QA datasets, a few-shot prompting strategy was employed, with representative examples built into the prompt. These followed a step-by-step explanation format and consisted of a concluding sentence beginning with the annotation **Conclusion:**. For claim verification, prompts were simply zero-shot, mainly due to the relative simplicity of the task as compared to question answering. Nevertheless, the overall of this sub-task was to elicit quality reasoning from LLMs in a format that could easily be decomposed into propositions, including the final conclusion step.

4.2.2 Reasoning Chain Construction using NLI

The raw, unprocessed text of an explanation provides little computational insight. Therefore, its format and structure must be formally defined before any quantitative evaluation is attempted. We do this by breaking down the explanations into intermediate propositions or "steps" at the sentence level. Each of these propositions is considered a node in the desired reasoning graph, with edges between the nodes representing logical and meaningful relations. Explanations are segmented into smaller reasoning units, typically at the level of sentences or clauses. Each unit is treated as a node in a semantic graph, with directed edges representing how the propositions logically and meaningfully connect to each other.

We use a lightweight natural language inference (NLI) model to derive said logical edges between the nodes of the graph. It determines whether a proposition follows another proposition within the same explanation. Eventually, these pairwise inferences are stored in an adjacency

matrix, which encodes the final structure of the reasoning graph. This adjacency matrix serves as a fundamental building block of our metric evaluation framework, and all metric calculations are based on this nuanced representation of an explanation’s logical structure. The key idea here is that every explanation is treated as a meaningful and ordered compilation of individual statements; statements that can exist independent of one another, yet be structured enough to help computationally determine the quality of said explanation.

Algorithm 2: Adjacency Matrix Construction

Input: Explanation $E = \{p_1, p_2, \dots, p_n\}$, NLI model M

Output: Adjacency matrix $A \in \mathbb{R}^{n \times n}$

Initialise A as a zero matrix;

for $i = 1$ *to* n **do**

for $j = 1$ *to* n **do**

 Infer relation $r = M(p_i, p_j)$;

 Encode r numerically (entailment, contradiction, neutral);

 Set $A[i, j] \leftarrow r$;

return A ;

Algorithm 2 clearly illustrates the procedure for constructing our required adjacency matrix with the help of a lightweight NLI model, namely, **bert-base-uncased-MNLI**. Given an explanation split into several propositions, these are then compared to other propositions in a pairwise manner to determine the logical flow (or "chain") of the explanation. These connections are then encoded numerically and stored in the adjacency matrix, where each entry $A[i, j]$ represents the relationship from proposition p_i to proposition p_j .

Component	Setting
CoT parsing max steps	20 (evaluator), 15 (intelligent-sampler cap)
MNLI model	textattack/bert-base-uncased-MNLI
Entailment threshold	0.85
Tokenization max length	512
NLI Batch size	16
Backend (prototype)	Ollama HTTP (/api/generate)
Output format	JSONL (per-sample), CSV/HTML (summary)

Table 4.1: Key hyperparameters and I/O conventions used in the CoT pipeline

Intelligent Sampling: While conducting our experiments, we found that some CoT explanations produced by the LLMs, especially Qwen, were excessively long, often comprising more than 15 propositions. Running natural language inference on such verbose explanations is computationally expensive. It requires pairwise entailment checks between all steps when constructing the adjacency matrix. For context, for an explanation with 20 steps, $20 \times 19 = 380$ entailment checks [$n \times (n-1)$ for n steps] are required to properly construct the reasoning chain! To address this, explanation propositions are pruned to a maximum of 15 steps using an intelligent sampling method. Steps are ranked by their position based on certain properties such as first & last priority, optimal length range, presence of reasoning markers (for example: *because*, *therefore*), and their logical connectivity to other propositions through shared entities. Properties such as meta-commentary (for example: *"let me think about it..."*) is penalized. The top fifteen steps are then selected (always including the first and last steps) and reordered chronologically. This ensures that long explanations are reduced to shorter, but semantically rich and representative subsets, mitigating the risk of running enormous NLI cycles.

4.2.3 Evaluation using Deductive Metrics

Now that the adjacency matrix is constructed, the explanations can be quantitatively assessed using well-defined and established metrics from [KT24]. As we refer to Chapter 3.1.1.1 of this

report, each metric describes a different property of the reasoning chain:

- **Weak relevance** measures whether each proposition maintains at least an indirect connection to the final conclusion answer or prediction label.
- **Strong relevance** determines whether a proposition is directly followed by the explanation’s final, predicted answer. High relevance scores represent structurally sound explanations.
- **Redundancy**, here, quantifies the extent to which a reasoning chain can be split into multiple components such that propositions from one of the components cannot connect to propositions from another component. Thus, high redundancy scores (closer to 1) indicate irrelevant propositions in the explanation that play no role in the central chain.

These metrics rely solely on the adjacency matrix to determine the seemingly latent relationships between the propositions. Algorithm 3 clearly represents the flow of the metric calculation defined in Chapter 3.1.1.1 of this report.

Algorithm 3: Evaluation of CoT Explanation Metrics

Input: Explanation $\mathcal{E} = \langle \mathcal{P}, \mathcal{R} \rangle$ with propositions $\mathcal{P}$ and directed edges $\mathcal{R}$, adjacency matrix $A \in \{0, 1\}^{n \times n}$ encoding $\mathcal{R}$, conclusion $\hat{y} \in \mathcal{P}$

Output: $m_{\text{red}}, m_{\text{weak}}, m_{\text{strong}}$

Define predicates and helpers:

$\text{DIRECT}(p_i, \hat{y}) \leftarrow \mathbb{K}[A[i, \hat{y}] = 1]$

$\text{PATH}(p_i, \hat{y}) \leftarrow \mathbb{K}[\exists \text{ a directed path } p_i \rightsquigarrow \hat{y} \text{ in } \mathcal{R}]$

$\text{COMPONENTS}(A) \leftarrow \text{list of weakly connected components of } \mathcal{R}$

$\text{MAIN}(A, \hat{y}) \leftarrow \text{the component returned by COMPONENTS that contains } \hat{y}$

1. Weak relevance

$$m_{\text{weak}} \leftarrow \frac{1}{|\mathcal{P}|} \sum_{p_i \in \mathcal{P}} \text{PATH}(p_i, \hat{y})$$

2. Strong relevance

$$m_{\text{strong}} \leftarrow \frac{1}{|\mathcal{P}|} \sum_{p_i \in \mathcal{P}} \text{DIRECT}(p_i, \hat{y})$$

3. Redundancy

$\mathcal{C} \leftarrow \text{COMPONENTS}(A)$

if $|\mathcal{C}| > 1$ **then** $m_{\text{red}} \leftarrow 1$

else $C_{\hat{y}} \leftarrow \text{MAIN}(A, \hat{y})$

$k \leftarrow |\{p_i \in C_{\hat{y}} \mid p_i \neq \hat{y}\}|$

$$m_{\text{red}} \leftarrow 1 - \frac{k}{|\mathcal{P}|}$$

return $\{m_{\text{red}}, m_{\text{weak}}, m_{\text{strong}}\}$

4.2.4 Aggregation and Reporting

Now that we finally have the computed metrics for all deductive explanations generated in our extensive experiments, the results are aggregated across datasets and models to obtain detailed insights. We look at results from both tasks: claim verification & question answering, metric-wise, model-wise, dataset-wise. Score heatmaps for each metric are generated, along with histograms and line graphs to better illustrate the performance of the models for different tasks. Detailed case studies and representative examples of the results are discussed in the next chapter.

4.3 ArgLLM pipeline

In this section, we talk about how our argLLM pipeline generates explanations in the form of an argumentation graph and evaluates them using metrics that capture both the structural properties of the graph and the semantics of the explanation; more specifically, the individual arguments.

4.3.1 Explanation Generation

For each input instance (question or claim), an LLM generates two bags of arguments, that is, two variants of an argLLM explanation: a *base* bag mined directly from the LLM, with each argument being assigned an neutral strength, and an *estimated* bag where each argument is additionally annotated with a model-assigned certainty (or uncertainty) and propagated strength under a gradual semantics.

The generation process, as demonstrated in Algorithm 4, can be divided into four steps: -

(A) Argument Mining

For every input X (in our case, claim or question), a (template) mining prompt Π_{mine} compels the model to generate an argument graph that comprises of: (i) a set of arguments $\mathcal{A} = \{a_i\}$, each with content text_i ; (ii) directed *support* links $\mathcal{R}_{\text{sup}} \subseteq \mathcal{A} \times \mathcal{A}$ (read " a_j supports a_i "); and (iii) directed *attack* links $\mathcal{R}_{\text{atk}} \subseteq \mathcal{A} \times \mathcal{A}$ (read " a_j attacks a_i "). We assign initial strengths s_i to each argument such that $s_i \leftarrow w_i^{\text{base}}$, where a neutral prior $w_i^{\text{base}} \in [0, 1]$ (e.g., 0.5). This constitutes our *base* bag of arguments $\mathcal{B}^{\text{base}} = (\mathcal{A}, \mathcal{R}_{\text{atk}}, \mathcal{R}_{\text{sup}})$, that will be propagated without assigning any uncertainty scores for the constituent arguments.

(B) Uncertainty Elicitation

We shift our attention to the other bag of arguments in question, $\mathcal{B}^{\text{est}}$. Given that individual arguments may vary in quality, it is useful to assign initial strengths to arguments based on their content. We use another prompt, Π_{cert} to extract a scalar uncertainty score for each generated argument a_i . The output is then parsed into $\hat{c}_i \in [0, 1]$, such that an estimated initial weight $w_i^{\text{est}} := \hat{c}_i$ can set the initial strength of the arguments to $s_i \leftarrow w_i^{\text{est}}$. Thus, the resultant bag $\mathcal{B}^{\text{est}}$ consists of the same tuple $(\mathcal{A}, \mathcal{R}_{\text{atk}}, \mathcal{R}_{\text{sup}})$, only with different initial strengths. Essentially, $\mathcal{B}^{\text{base}}$ is about graph topology, while $\mathcal{B}^{\text{est}}$ takes into account the relative quality of arguments.

(C) Strength Propagation (Gradual Semantics)

The two generated bags of arguments $\mathcal{B}^{\text{base}}$ and $\mathcal{B}^{\text{est}}$ are then evaluated under the same gradual semantics. Let $S_i^+ = \{j \mid (a_j, a_i) \in \mathcal{R}_{\text{sup}}\}$ and $S_i^- = \{j \mid (a_j, a_i) \in \mathcal{R}_{\text{atk}}\}$. Multiple parents are combined via the *aggregation* operator **Agg** (refer Algorithm 4), the node updates are modulated with the help of an *influence* operator **Inf**. Mathematically,

$$\text{Agg}_{\Pi}(U) = \prod_{u \in U} u \quad (\text{product aggregation}), \quad \text{Agg}_{\Pi^-}(U) = \prod_{u \in U} (1 - u),$$

$$s_i \leftarrow \text{Inf}(w_i, \text{Agg}_{\Pi}(\{s_j : j \in S_i^+\}), \text{Agg}_{\Pi^-}(\{1-s_j : j \in S_i^-\})) = \alpha w_i + (1-\alpha) (\text{Agg}_{\Pi}(S_i^+) \cdot \text{Agg}_{\Pi^-}(S_i^-)),$$

where $\alpha \in [0, 1]$ is a conservativeness parameter (e.g., $\alpha = 1/(1 + \lambda)$ for some $\lambda \geq 0$). This is iterated (simultaneously) for all nodes until convergence or for a small fixed horizon T (usually < 10). Long chains are penalized during product aggregation, and linear influence blends the node's own weight with relational evidence. It is worthwhile to note that for stronger attackers to minimize the target's update multiplicatively, the attack term uses $(1 - s_j)$.

(D) Bag-level Prediction

Let a_{root} denote the designated conclusion node. Following the final iteration, the bag-level score is the root strength:

$$s_{\text{root}}^{\text{base}} := s(a_{\text{root}}; \mathcal{B}^{\text{base}}), \quad s_{\text{root}}^{\text{est}} := s(a_{\text{root}}; \mathcal{B}^{\text{est}}).$$

Both bags of arguments are then subjected to evaluation under structural and semantic metrics.

Note: We also log argument counts and degree statistics to contextualize scores.

Algorithm 4: argLLM Generation + Gradual Semantics Propagation

Input: Input text X (question or claim), LLM back-end $\mathcal{M}$, prompts $\Pi_{\text{mine}}, \Pi_{\text{cert}}$

Output: Two bags $\mathcal{B}^{\text{base}}, \mathcal{B}^{\text{est}}$ with prediction scores $s_{\text{root}}^{\text{base}}, s_{\text{root}}^{\text{est}}$

(A) Mine arguments and relations

Query $\mathcal{M}$ with $\Pi_{\text{mine}}(X)$ to obtain candidate sets $\mathcal{A}$, attacks $\mathcal{R}_{\text{atk}}$, supports $\mathcal{R}_{\text{sup}}$.

Assign default initial weights w_i^{base} (e.g., 0.5) and set strengths $s_i \leftarrow w_i^{\text{base}}$.

Form $\mathcal{B}^{\text{base}} \leftarrow (\mathcal{A}, \mathcal{R}_{\text{atk}}, \mathcal{R}_{\text{sup}})$.

(B) Estimate per-argument certainty

For each $a_i \in \mathcal{A}$, query $\Pi_{\text{cert}}(\text{text}_i)$ to elicit a scalar certainty $\hat{c}_i$; robustly parse to $[0, 1]$.

Set $w_i^{\text{est}} \leftarrow \hat{c}_i$ and initialize $s_i \leftarrow w_i^{\text{est}}$. Construct $\mathcal{B}^{\text{est}}$ with the same edges.

(C) Propagate strengths (gradual semantics)

Choose aggregation Agg (e.g., product) and influence Inf (e.g., linear). For $t = 1 \dots T$ or until convergence:

foreach a_i **do**

$S_i^+ \leftarrow \{j \mid (a_j, a_i) \in \mathcal{R}_{\text{sup}}\}, \quad S_i^- \leftarrow \{j \mid (a_j, a_i) \in \mathcal{R}_{\text{atk}}\}$
 $s_i \leftarrow \text{Inf}(w_i, \text{Agg}(\{s_j : j \in S_i^+\}), \text{Agg}(\{1-s_j : j \in S_i^-\}))$

(D) Read out predictions

$s_{\text{root}}^{\text{base}} \leftarrow s_{\text{root}}(\mathcal{B}^{\text{base}}), \quad s_{\text{root}}^{\text{est}} \leftarrow s_{\text{root}}(\mathcal{B}^{\text{est}}).$

return $(\mathcal{B}^{\text{base}}, s_{\text{root}}^{\text{base}}, \mathcal{B}^{\text{est}}, s_{\text{root}}^{\text{est}})$

Visualizing the Explanation: Argument Graph

Each explanation is a *bag* $\mathcal{B} = (\mathcal{A}, \mathcal{R}_{\text{atk}}, \mathcal{R}_{\text{sup}})$ with arguments $\mathcal{A} = \{a_i = (\text{id}_i, \text{text}_i, w_i, s_i)\}$, where id_i is the argument ID, text_i is the argument content, w_i is the initial weight (e.g., certainty) and s_i the propagated strength; $\mathcal{R}_{\text{atk}} \subseteq \mathcal{A} \times \mathcal{A}$ are attack relations and $\mathcal{R}_{\text{sup}} \subseteq \mathcal{A} \times \mathcal{A}$ support relations. The root a_{root} of the argumentative explanation represents the central claim in question. A bag's prediction is the strength s_{root} after propagation, which is then thresholded (over 0.5) to generate a discrete label prediction. Listing 4.5 clearly presents this explanation structure. At depth $D = 1$ and breadth $B = 1$, claim *db0* has: (a) a supporting argument *Sdb0* such that $Sdb0 \rightarrow db0$; and (b) an attacking argument *Adb0* such that $Adb0 \rightarrow db0$.

```
{
  "question": "...",
  "claim": "...",
  "base": {
    "bag": {
      "arguments": { "db0": {"argument": "...", "initial_weight": 0.50, "strength": 0.25},
                    "Sdb0": {"argument": "...", "initial_weight": 0.45, "strength": 0.45},
                    "Adb0": {"argument": "...", "initial_weight": 0.95, "strength": 0.95}
                  },
      "attacks": [["Adb0", "db0"]],
      "supports": [["Sdb0", "db0"]],
    },
    "prediction": 0.25
  },
  "estimated": { ... } // same schema, but with certainty-informed weights
}
```

Listing 4.5: Example explanation produced by the generator script

4.3.2 Evaluation using Argumentative Metrics

Next, the generated explanations are extensively tested using properties and metrics defined in [KT24], namely *Acceptability* and *Circularity*. To comprehensively evaluate the robustness of these explanations, we realize two interpretations of these metrics: *structural* and *semantic* as discussed in Chapter 3 of this report. The structural perspective simply emulates the definitions of these

metrics from [KT24] and measures the quality of explanations solely on the basis of their abstract structures. The semantic perspective, on the other hand, takes the evaluation beyond the mere format of reasoning, taking into account the actual textual content of the arguments along with their structure.

Algorithm 5 illustrates the evaluation process using the structural metrics. Each argLLM bag is represented as a directed, signed graph, and then normalized such that support edges share a single orientation. The predicted conclusion $\mathcal{Y}$ is chosen by the root selector σ . We then measure: (a) *circularity*, by counting the number of cycles over the union of attack/support edges and dividing by $|A|$; and (ii) *acceptability* by verifying whether attackers of each node are further "counter-attacked" by their own attackers. In other words, *is the claim left undefended?* Output rows record argument counts (supporters and attackers), and the two metric scores, $\text{Circ}(G_b)$ & $\text{Acc}(G_b)$.

Algorithm 5: Structural Evaluation for an argLLM Output File

Input: Records $\{\mathcal{R}_k\}$, chosen bags $\text{BAGS} \subseteq \{\text{base}, \text{estimated}\}$, root selector $\sigma \in \{\text{db0}, \text{root}, \text{auto}\}$
Output: Row-wise table of structural metrics (counts, Circ, Acc)

```

foreach record  $\mathcal{R}$  do
  foreach bag  $b \in \text{BAGS}$  do
    Parse the bag as a directed, signed graph  $G_b = \langle A, R_{\text{sup}}, R_{\text{att}} \rangle$ ;
    // Unify edge orientation for support so that  $(u, v) \in R_{\text{sup}}$  means “ $u$ 
      supports  $v$ ”
    If supports are given as (supporter, supported), rewrite as (supported, supporter);
    // Root (conclusion) selection
    if  $\sigma = \text{db0}$  and  $\text{db0} \in A$  then
      |  $\mathcal{Y} \leftarrow \{\text{db0}\}$ 
    else if  $\sigma = \text{root}$  then
      |  $\mathcal{Y} \leftarrow \{\arg \max_{a \in A} \deg_{G_b}^-(a)\}$ 
    else auto
      |  $\mathcal{Y} \leftarrow \{\text{db0}\}$  if present, else as in root
    // Structural metrics on  $G_b$ 
    Detect cycles in  $R_{\text{sup}} \cup R_{\text{att}}$  and set  $\text{Circ}(G_b) = \frac{\# \text{cycles}}{|A|}$ ;
    For acceptability, for each  $y \in \mathcal{Y}$  and its defenders  $D(y) = \{u : (u, y) \in R_{\text{sup}}\}$ ,
      compute  $\text{Acc}(G_b) = \frac{1}{|A|} \sum_{a \in A} \frac{1}{|\text{Atts}(a)|} \sum_{(j, a) \in R_{\text{att}}} \mathbf{1}[\text{Atts}(j) \neq \emptyset]$ ;
    Emit row with:  $|A|, |R_{\text{att}}|, |R_{\text{sup}}|, \mathcal{Y}, \text{Circ}(G_b), \text{Acc}(G_b)$ .
  return concatenated table (CSV)

```

As we go beyond the structural interpretations of Acceptability and Circularity, we consider what the arguments actually say and measure their relative similarities. We begin by embedding all unique texts once and later reusing all cached vectors. Semantic Circularity checks whether supporters that paraphrase the claim, which is determined with the help of cosine similarity scores between the claim and the supporter. A similarity greater than the threshold $\geq \tau_{\text{circ}}$ is flagged and counted. Semantic Acceptability consider an attacker "neutralized" if it is semantically similar to the claim or any supporter of the claim (cosine similarity $\geq \tau_{\text{neu}}$). We average these counts to yield SCirc and SAcc, giving discriminative and robust diagnostics that add rich context to the structural scores. Algorithm 6 highlights the implementation of this process in detail.

Algorithm 6: Semantic Evaluation with Shared Embeddings

Input: Records $\{\mathcal{R}_k\}$, bags BAGS, thresholds $(\tau_{\text{circ}}, \tau_{\text{neu}})$, sentence encoder ϕ

Output: Row-wise table with semantic metrics SCirc, SAcc

Pass 1 (scan): For all k and $b \in \text{BAGS}$, collect the set of unique texts

$\mathcal{T} = \{\text{text}(a) : a \in A_b\}$.

Pass 2 (embed-once): Compute a cache $E : \text{text} \mapsto \mathbf{e} = \phi(\text{text}) / \|\phi(\text{text})\|_2$;

Pass 3 (per record/bag scoring): **foreach** $\mathcal{R}$, $b \in \text{BAGS}$ **do**

 Extract $G_b = \langle A, R_{\text{sup}}, R_{\text{att}} \rangle$ and root r (as in Alg. 5);

 Let $S(r) = \{s : (s, r) \in R_{\text{sup}}\}$ and $A^-(r) = \{a : (a, r) \in R_{\text{att}}\}$;

 // Semantic circularity: supporters should bring new information

 With cosine similarity $\text{sim}(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{y}$, compute

$$\text{SCirc}(G_b) = \begin{cases} \frac{1}{|S(r)|} \sum_{s \in S(r)} \mathbf{1}[\text{sim}(E[s], E[r]) \geq \tau_{\text{circ}}], & |S(r)| > 0, \\ 0, & \text{otherwise.} \end{cases}$$

 // Semantic acceptability: attackers neutralized if too close to claim
 or a supporter

 For each $a \in A^-(r)$ let $m(a) = \max(\text{sim}(E[a], E[r]), \max_{s \in S(r)} \text{sim}(E[a], E[s]))$;

 Set

$$\text{SAcc}(G_b) = \begin{cases} 1, & |A^-(r)| = 0, \\ \frac{1}{|A^-(r)|} \sum_{a \in A^-(r)} \mathbf{1}[m(a) \geq \tau_{\text{neu}}], & \text{otherwise.} \end{cases}$$

 Emit row with counts $(|A|, |S(r)|, |A^-(r)|)$ and (SCirc, SAcc).

return dataset/model-specific CSVs and a styled HTML summary (QA and Claim-Verification tables)

4.4 The SHAKE test pipeline

In this section, we will discuss the implementation of our new faithfulness evaluation test: the SHAKE test. Let us recall the key idea that fuels this attention perturbation test. For faithfulness to be satisfied, an LLM should return only such an explanation that has actually influenced its decision-making when responding to a certain prompt. So, to test this, we prompt the model to return certain rationale tokens (also referred to as "keywords") from the input claim that it believes was important in reaching the final answer. Consequently, we zero out the model's attention to those keywords. We hypothesize that if the model's prediction label flips ($\text{TRUE} \leftrightarrow \text{FALSE}$), then the explanation is faithful to the model internal reasoning and implicit knowledge base; else, the explanation is considered to be unfaithful. In our codebase for this evaluation pipeline, we implement attention perturbation backends for models from the Llama, Mistral, and Qwen families, as well as a batch runner that projects the flip entries in a CSV file.

4.4.1 Overview

Figure 4.2 shares a brief overview of the entire attention perturbation architecture. We begin by prompting the model to predict whether a given claim is TRUE or FALSE, based on its understanding of the claim and its own internal knowledge. Simultaneously, we ask the model to output upto 4 "keywords" that it believes played an important role in the initial prediction. These rationale tokens are then mapped to their exact subword indices for alignment. In the *perturbation* step, we first identify the last L attention layers of the model, and then replace them with custom attention modules that specifically zero out attention to those very rationale tokens, followed by renormalization of the updated attention weights. Finally, we check whether the label has flipped after the perturbation. If it has, we record a Binary Flip Score of 1 (faithfulness satisfied); else we record a Binary Flip Score of 0 (unfaithful).



Figure 4.2: End-to-end SHAKE pipeline (generalized for model’s last L attention layers)

4.4.2 Rationale Token Selection & Alignment

At the heart of our SHAKE pipeline is the rationale token or keyword extraction step. These tokens carry a lot of intuitive meaning in our hypothesis. The argument is simple: if enquired, does the model return a valid and true explanation about how and why it made a certain decision regarding a question (or claim)? Therefore, it is crucial to carefully design the prompt that elicits these all-important keywords.

We explored a number of different strategies when designing this prompt. One of the ideas was to ask the model to generate a large natural language explanation highlighting its reasoning and then extract relevant keywords from the text. However, it was observed that the contexts in which those keywords were used in the explanations were irrelevant and could not be considered as appropriate rationale tokens to be used in the attention perturbation process. Recalling Occam’s razor, which states that between competing solutions, the simplest one with the least complexity is often the best option, we decided to proceed with a simple, explicit prompt that directly asks the LLM to return keywords from the input claim statement itself. The prompt structure is illustrated in listing 4.6. It is an efficient combination of system and user prompts.

```

messages = [
  {
    "role": "system",
    "content": "You are a keyword extractor. Respond only with the requested words,
               nothing else.",
  },
]
  
```

```

{
  "role": "user",
  "content": (
    f'Claim: "{text}"\n\n'
    "Extract 1-4 key words from this claim. Respond with only the words separated
      by spaces, no explanations."
  ),
},
]

```

Listing 4.6: The prompt structure for explanation token or "Keyword" extraction. A number of different prompt structures were tested: in one instance, we prompted the model to simply print words that it deemed as important for its answer. However, the responses were vague and often abundant with fillers. Finally, this combination of user and system prompts generated valid keywords that apparently reflected the model's reasoning.

Subword Alignment: Since attention operates at the subword level, it is important to align rationale words to the corresponding token indices computed from the input claim string. So, we use a tokenizer to obtain character spans per token and then match the relevant lowercase rationale words to the lowercase claim string with strict word-boundary checks. Tokens whose character spans overlap the matched word span are marked as "targets" which the model should not look at after attention perturbation. Algorithm 7 highlights how this is logically implemented.

Algorithm 7: Subword Alignment for Explanation/Rationale Words

Input: Claim string c , tokenizer T , rationale words $\mathcal{W} = \{w_k\}$

Output: Target subword indices $\mathcal{I}$

```

enc ←  $T(c, \text{return\_offsets\_mapping} = \text{True}, \text{add\_special\_tokens} = \text{False})$ 
 $S \leftarrow \text{enc.offset\_mapping}$  // character spans per token
 $\mathcal{I} \leftarrow []$ 
for  $w \in \mathcal{W}$  do
   $w' \leftarrow \text{lowercase}(w)$ ;  $c' \leftarrow \text{lowercase}(c)$ 
  Find first word-boundary match  $(s, e)$  of  $w'$  in  $c'$ ;
  for  $t = 1$  to  $|S|$  do
    if  $S[t]$  overlaps  $(s, e)$  then append  $t$  to  $\mathcal{I}$ 
return  $\text{Unique}(\mathcal{I})$ 

```

4.4.3 Attention Hooks & Perturbation

We perform attention perturbation on models from the LLaMA, Mistral, and Qwen families. The variants to be used had to be carefully picked, owing to the fact that each of these models needed to have open weights, allowing us to probe into individual layers and override the existing attention modules (as provided by the HuggingFace `transformers` library) with custom built ones. Therefore, for each model backend, we set `output_attentions=True` which allows us to view the original attention weights given to the tokens. These weights are then stored for future reference (example, baseline comparisons). Finally, all attention weights corresponding to our target rationale tokens are zeroed out across every head, preventing the model from looking at those keywords when making a decision. The modified attention scores are then renormalized so that the remaining tokens share "importance" across properly.

It is important to note that the implementation is generalized to apply this perturbation to the attention modules of the final L attention layers. However, we will discuss in the next section why choosing $L = 1$ has proved to be optimal. This design preserves the general behavior of the pre-trained LLM, while only selectively preventing it from looking at rationale tokens in the deepest layers where final predictions are actually made. Algorithm 8 accurately describes the perturbation process.

Algorithm 8: Perturb: Forward-Pass Attention Zeroing (per selected layer)

Input: Hidden states H , attention mask M , target key indices $\mathcal{I}$
Output: Attended output O , (original, perturbed) attention weights ($A^{\text{orig}}, A^{\text{pert}}$)

$(O, A) \leftarrow \text{BASEATTENTIONFORWARD}(H, M, \text{output_attentions}=\text{True})$
// $A \in \mathbb{R}^{B \times H_d \times Q \times K}$
if A^{orig} not set **then** $A^{\text{orig}} \leftarrow A$
for $i \in \mathcal{I}$ **do**
 $_ \text{set } A[:, :, :, i] \leftarrow 0$
 $Z \leftarrow \sum_k A[:, :, :, k]$ // sum over key dimension
 if $Z = 0$ at any position **then** set corresponding row to uniform over keys
 $A \leftarrow A / (Z + \epsilon)$ // renormalize
 $A^{\text{pert}} \leftarrow A$
return ($\text{APPLY}(A, V)$, $A^{\text{orig}}, A^{\text{pert}}$)

4.4.4 Decision & Scoring

Once we have modified the required attention modules of the concerned model, we prompt the model, once again, with the original input claim. This type of counterfactual or ablation test performed internally, in the latent space is useful as it does not alter the semantics of the input sentence. It does not lead to any unnatural inputs being fed to the model, thereby allowing a thorough evaluation of the faithfulness property.

We define a binary flip score to quantify faithfulness. Let $y_{\text{orig}} \in \{\text{TRUE}, \text{FALSE}\}$ be the unperturbed label and $y_{\text{pert}} \in \{\text{TRUE}, \text{FALSE}\}$ be the perturbed label. The *Binary Flip Score (BFS)* is:

$$\text{BFS} = \begin{cases} 1, & \text{if } y_{\text{orig}} \neq y_{\text{pert}}, \\ 0, & \text{otherwise.} \end{cases}$$

There are of course, outliers and exceptions in the produced labels. Both, y_{orig} and y_{pert} are sometimes unable to yield any valid labels and in such cases the output CSV field is populated with UNKNOWN. When computing dataset-level faithfulness, such entries are ignored.

4.4.5 End-to-End Testing

Finally, bringing all of the components together, we write a runner script that iterates over an entire dataset of claims, and for each sample: (a) elicits the base label y_{orig} from the model, (b) elicits upto 4 keywords W from the model, (c) aligns these keywords to the appropriate subword indices from the claim statement, (d) replaces the last attention layer $L = 1$ with the custom attention module, (e) elicits the perturbed label y_{pert} from the model, under identical decoding settings, and (f) captures the *BFS* into a CSV file, along with the corresponding `sample_id`, `claim`, `ground_truth`, and rationale tokens. Algorithm 9 demonstrates how this end-to-end evaluation is executed.

Algorithm 9: SHAKE Test Runner Script

Input: Claims $\{(i, c_i, y_i^{\text{gold}})\}$, model $\mathcal{M}$, tokenizer T , last- L layers
Output: Table with `sample_id`, `claim`, y_{orig} , y_{pert} , y_i^{gold} , rationale tokens, BFS

for each $(i, c_i, y_i^{\text{gold}})$ **do**
 $y_{\text{orig}} \leftarrow \text{LABEL}(c_i)$ // unperturbed
 $W \leftarrow \text{EXTRACTKEYWORDS}(c_i)$ // 1-4 words
 $\mathcal{I} \leftarrow \text{ALIGNTOSUBWORDS}(c_i, T, W)$ // offset mapping + word boundaries
 if $\mathcal{I} = \emptyset$ **then** $y_{\text{pert}} \leftarrow \text{UNKNOWN}$; **goto** record
 $\text{REPLACELASTLAYERS}(\mathcal{M}, \mathcal{I}, L)$ $y_{\text{pert}} \leftarrow \text{LABEL}^{\text{pert}}(c_i)$
 $\text{BFS} \leftarrow \mathbb{I}[y_{\text{orig}} \in \{\text{T}, \text{F}\} \wedge y_{\text{pert}} \in \{\text{T}, \text{F}\} \wedge y_{\text{orig}} \neq y_{\text{pert}}]$
 $\text{APPENDROW}(i, c_i, y_{\text{orig}}, y_{\text{pert}}, y_i^{\text{gold}}, W, \text{BFS})$
 $\text{WRITECSV}(); \text{WRITEHTML}();$ **return** outputs

Models Used & Output Format

As discussed, we used models from the Llama, Mistral, Qwen families, namely, LLaMA-2 7B chat (meta-llama/Llama-2-7b-chat-hf), Mistral-7B Instruct (mistralai/Mistral-7B-Instruct-v0.1), and Qwen 2.5 7B Instruct (Qwen/Qwen2.5-7B-Instruct).

All output were systematically recorded in a CSV file with the following schema:

```
sample_id, claim, label_orig, label_pert, ground_truth, rationale_tokens, bfs.
```

Additional Experiments: Perturbing Tokens From CoT Explanations, $SHAKE_{expl}$

We also explored a variant of the SHAKE pipeline where rationale tokens were extracted from Chain-of-Thought generated explanations. The key idea was to first prompt the model to produce a step-by-step reasoning about the input claim using a few-shot/zero-shot CoT prompt, without generating the final prediction label. This would be followed by passing both the claim and the newly generated explanation to the LLM and prompting it to now generate the claim. Simultaneously, another prompt would elicit the rationale tokens, this time from the CoT explanation. The model’s attention to these tokens/keywords would then be perturbed (zeroed out), followed by one final pass through the model of the claim and the explanation. Again, if the label flips after the perturbation process, it would be considered faithful; else unfaithful. This method too, was based on the idea that if the explanation truly reflects the model’s belief, then perturbing parts of it should overturn the final decision.

Methodology: For each input claim c we (a) prompted the model to produce a multi-step CoT explanation e , without producing a prediction label; (b) passed both the claim and the explanation to the model to elicit the base label $y_{base} \in \{\text{TRUE}, \text{FALSE}\}$; (c) asked the LLM to return K rationale tokens from explanation e that truly influenced its decision; (d) replaced the attention modules of the last L layers with custom modules that had zero-weights at the rationale token indices, and renormalized the weights; and (e) ran the same (c, e) prompt through the model to extract the perturbed prediction label and finally compute the binary flip score as,

$$\text{BFS}_{\text{expl}} = \mathbb{I}[y_{base} \neq y_{\text{pert}}],$$

In the next section, we will discuss our findings from the CoT, argLLM, original SHAKE pipeline in detail and decipher the underlying patterns observed in the results, one by one.

4.5 Pilot User Study

As described in Chapter 1 of our report, we conducted a pilot user study to understand the human perspective on LLM-generated explanations and assess whether the computational evaluation of these explanations tally with human annotations. We try to draw a relationship between satisfaction of properties (redundancy, weak and strong relevance, acceptability, circularity) and human intuitions about the explanations. The study is intended to serve as a preliminary feasibility test based on the methodologies described in [BKPW23], [BDM⁺24], and can be the stepping stone to a larger, more comprehensive study with explanations generated from other larger models.

The study was performed on a small group of 10 participants, from diverse backgrounds, who were frequent users of different LLMs, either for study or work. Only a couple of participants were involved in the field of AI development. This was deliberately done to prevent existing knowledge bias when answering questions about usefulness of LLM explanations and allowed us to assess how the explanations were perceived by non-specialists, incorporating a more generalized view. Asking participants to rate explanations on abstract properties such as “weak relevance” or “circularity” (with textbook definitions explaining the meaning of each property) would be an assessment of the

interpretability of the definitions of properties rather than the interpretability of the explanations themselves. Therefore, we concluded on a more generalized, simple set of questions that asked whether the explanations were convincing and/or useful for them to understand and reason about the claim or question, and to what extent. This allowed for a seamless comparison between the degree of human acceptance and computational measures that represent the characteristic properties of these explanations.

4.5.1 Design

The survey was conducted using the Google Forms platform. A comprehensive form was constructed and participants were asked for:

- **Introduction and Consent:** A simpler identifier such as their first name or initials, allowed us to track and map responses to the participant; additionally, we asked for the background of the user to ensure that diverse areas of expertise were represented.
- **Item evaluation:** A set of 6 claim/question-explanations pairs were sampled randomly from the two explanation paradigms, representing different models and datasets. For each pair, we (i) displayed the content of claim (or question) and explanation, and (ii) asked a set of trivial questions to the users, based on their understanding of the explanation. These were,
 - *Q1 “Do you buy this explanation?”* (radio button options: Yes/No/Unsure) - captures the overall acceptance signal of the explanation, in the most basic sense
 - *Q2 “How convincing is this explanation?”* (1-5 Likert scale) - captures the ability of the explanation to convince the user about the polarity of the claim (or the answer to the question)
 - *Q3 “How clear is this explanation?”* (1-5 Likert scale) - captures the level clarity the reasoning provides when explaining its thinking
 - *Q4 “Any issues you noticed?”* (optional paragraph field) - retains any other views the user has about the explanation in question

Data Format: Responses were recorded in Google Sheets and the schema used to store the data is summarised in Table 4.2.

Table 4.2: Schema for user study data export

Column	Type	Description
participant_id	string	First name or initials provided by participant
participant_bg	string	Self-reported background of the participant
item_order	integer	Serial number of the item
item_id	string	Unique identifier for the item
claim	string	Randomly-sampled claim or question item as shown to the participant
explanation	string	Corresponding explanation for the item (in full)
buy	categorical	Participant’s alignment with the explanation: Yes / No / Unsure
conv	integer (1–5)	Whether the participant is convinced; rating (1=not convincing, 5=extremely convincing)
clar	integer (1–5)	Clarity rating (1=very unclear, 5=very clear)
free_text	string	Optional comments for issues identified by participant

Chapter 5

Results & Evaluation

Now, let us look at the results of our experiments conducted using different pipelines to evaluate the explanation paradigms in question. To understand the quality of the explanations generated by CoT & argLLMs, we are more concerned about the results generated using the evaluation metrics discussed in the Implementation section of this report. In other words, this section stresses more on the quality of explanations rather than the accuracy of answers generated by the LLMs. However, we also shed some light on the relationship between the accuracy of predictions and the relative satisfaction of properties of explanations. When it comes to our novel SHAKE test, we look at the faithfulness results produced by the attention perturbation mechanism and compare them to the Binary Flip Scores (BFS) after perturbing the model’s attention to random tokens.

5.1 Chain-of-Thought (CoT) Pipeline

As discussed, we evaluated the explanations generated by LLMs using the CoT prompting strategy across multiple reasoning datasets. The explanations were generated using consistent few-shot and zero-shot Chain-of-Thought prompting templates. We recorded the relevant evaluation metrics for the deductive explanations and further analyzed them on a per-model, per-metric basis. Before we dive into the analysis of our results, let us recap what the results mean and how to read them. Redundancy scores are ideal when they are close to 0, that is, the lower the better. Near-1.0 scores indicate that the explanation consists of steps that do little to justify the final answer. Weak Relevance and Strong Relevance scores, for a "good" CoT explanation, should be closer to 1.0 than 0. Thus, these explanations consist of steps that are directly or indirectly followed by the answer.

5.1.1 Claim Verification

For claim verification, Table 5.1 summarizes the final *Weak Relevance*, *Strong Relevance*, and *Redundancy* scores, for *Mistral 7B Instruct*, *Meta Llama3 8B*, and *Qwen3 8B* models. It can be observed that redundancy is very high (>0.93) across models and reasoning tasks, indicating that CoT reasoning chains can be split into multiple disconnected components, with intermediate propositions that have little to no impact on the final outcome. These can be attributed to limitations such as lengthy and verbose explanations, especially in those generated by *Qwen*. After closely observing some of the explanations generated by these models, it was found that explanations generated for claims from the MedClaim dataset are particularly comprised of a lot of "extra talk" such as restating the claim and filler texts (e.g., "Let me think about it..."), further contributing to the high redundancy measures in this domain.

Relevance scores, on the other hand, are at the lower end of the spectrum, indicating a disconnect between the intermediate reasoning steps and the final conclusion. Weak Relevance (relaxed condition) ranges 0.13–0.27, while Strong Relevance (stricter condition) scores are relatively lower at 0.11–0.23. Intuitively, this delta is logical, as Strong Relevance requires propositions to directly connect to the conclusion, while Weak Relevance allows an indirect link.

Mistral emerges as the best and most robust model among the three. Produces the most relevant and least redundant explanations across datasets, with the following macro-averages across all

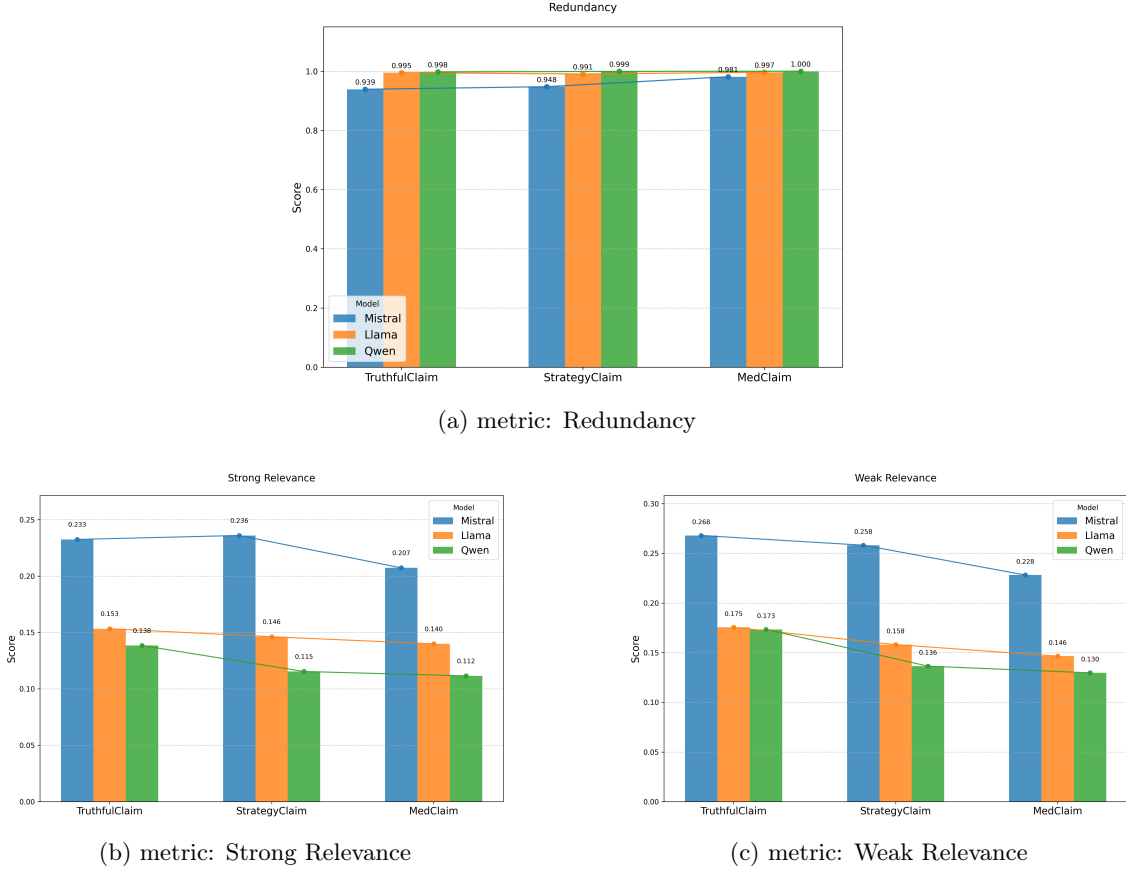


Figure 5.1: Metric-wise plots for Mistral, Llama, and Qwen models: Claim Verification

Table 5.1: CoT explanation metrics on claim verification datasets. Arrows indicate preferred direction. Best per-row values are **bold**.

Dataset	Mistral			Llama			Qwen		
	Red. ↓	Weak Rel. ↑	Strong Rel. ↑	Red. ↓	Weak Rel. ↑	Strong Rel. ↑	Red. ↓	Weak Rel. ↑	Strong Rel. ↑
Truthfulclaim	0.9391	0.2680	0.2325	0.9951	0.1755	0.1533	0.9978	0.1734	0.1384
Strategyclaim	0.9478	0.2582	0.2360	0.9906	0.1581	0.1462	0.9994	0.1364	0.1154
Medclaim	0.9814	0.2282	0.2073	0.9966	0.1465	0.1399	1.0000	0.1297	0.1115
Macro-avg	0.9561	0.2515	0.2253	0.9941	0.1600	0.1465	0.9991	0.1465	0.1218

three metrics: (a) redundancy: 0.9561 (vs. 0.9941 for Llama; 0.9991 for Qwen); (b) weak relevance: 0.2515 (vs. 0.1600 for Llama; 0.1465 for Qwen); and (c) strong relevance: 0.2253 (vs. 0.1465 for Llama; 0.1218 for Qwen).

From another lens, MedClaim turned out to be the hardest dataset to generate reasoning for. This is primarily due to the elaborate and vast nature of the sample claims. Another reason could be the models' lack of domain-specific proficiency. These general-purpose models could be struggling to reason effectively about claims from the medical field. Across models, it demonstrates the highest redundancy scores and the lowest relevance, with Qwen hitting absolute redundancy at 1.0000. Figures 5.1 and 5.2 illustrate the above trends, with the help of detailed plots and heatmaps.

Evaluation Metrics vs. Prediction Accuracy Table 5.2 provides an interesting perspective on the relationship between prediction accuracies and satisfaction of properties of the explanations. While the Mistral model has the most intrinsically "good" explanations, it usually performs worse than Qwen in terms of the prediction accuracy of labels. So, while Qwen mostly produces redundant explanations, it records the highest accuracies on the TruthfulClaim and MedClaim datasets. This indicates that plausible-sounding explanations may not always result in correct answers. A model

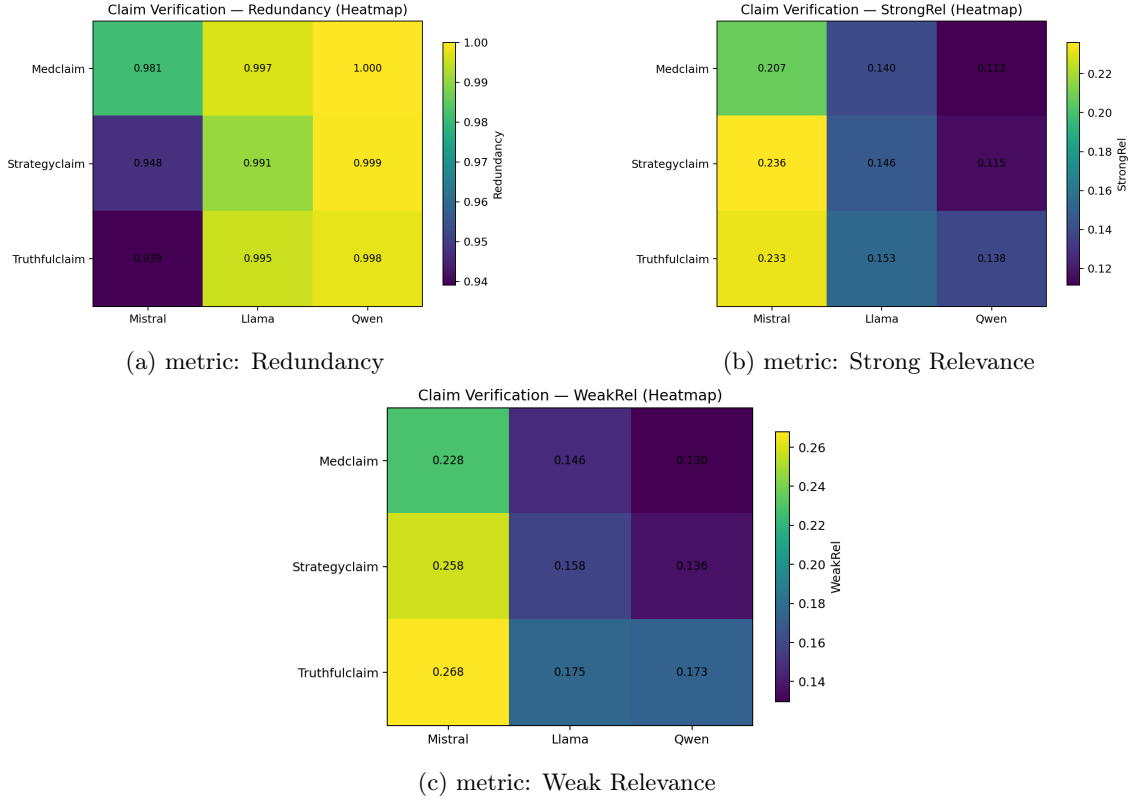


Figure 5.2: Metric-wise heatmaps for Mistral, Llama, and Qwen explanations: Claim Verification

Table 5.2: Prediction accuracy of CoT explanations across datasets and models.

Dataset	Mistral	Llama	Qwen
Truthfulclaim	0.622	0.615	0.650
Strategyclaim	0.432	0.326	0.361
Medclaim	0.482	0.532	0.607

can produce the correct answers yet have extremely verbose and irrelevant explanations. This further deems the exploration of newer ways to define and measure faithfulness of explanations as generated by large language models.

5.1.2 Question Answering

The results for Question Answering tasks follow a similar trend, though absolute numbers differ. Redundancy scores remain high, ranging from 0.92 all the way upto 1. Once again, the Qwen3 8B model’s redundancy is maximal (1.00) on the TruthfulQA and StrategyQA datasets, which is logical owing to its tendency to generate extremely verbose reasoning structures with disconnected intermediates. Interestingly, while the results of MedQA from the Mistral and Llama models are in line with the notion of domain incompetency, Qwen performs surprisingly better on MedQA than its Truthful and Strategy counterparts.

It can be observed that models perform better on relevance measures for QA than claim verification. Both Strong and Weak Relevance have higher maximum values this time, plus there is a significant gap between their respective ranges (0.13–0.35 for strong relevance; 0.11–0.29 for weak relevance). This gap suggests that even though many propositions are not truly evidential of the answer, they are definitely about the topic of the question.

Once again, Mistral emerges as the winner, the best model across all QA datasets, in all metrics. The macro-averages demonstrate a substantial overall difference in performance between Mistral and the other two models, especially in terms of Weak Relevance and Strong Relevance.

Table 5.3: CoT explanation metrics on QA datasets. Arrows indicate preferred direction. Best per-row values are **bold**.

Dataset	Mistral			Llama			Qwen		
	Red. ↓	Weak Rel. ↑	Strong Rel. ↑	Red. ↓	Weak Rel. ↑	Strong Rel. ↑	Red. ↓	Weak Rel. ↑	Strong Rel. ↑
TruthfulQA	0.9189	0.3481	0.2942	0.9792	0.2178	0.1994	1.0000	0.1490	0.1198
StrategyQA	0.9234	0.3049	0.2770	0.9706	0.2044	0.1928	1.0000	0.1287	0.1092
MedQA	0.9878	0.2528	0.2243	0.9884	0.1706	0.1582	0.9993	0.1380	0.1183
Macro-avg	0.9434	0.3019	0.2652	0.9794	0.1976	0.1835	0.9998	0.1386	0.1158

The results over two reasoning paradigms reveal that Chain-of-Thought explanations usually tend to be very long and repetitive, with constituent reasoning steps often being irrelevant to the topic at hand, or even the final outcome. These characteristics can be attributed to an observation discussed earlier in Chapter 2 of our report. [WWS⁺23] highlights that the effectiveness of CoT explanations vary with the model scale. Specifically, LLMs with less than 100 billion parameters tend to generate poor quality explanations. Since these results come from models that are merely 7-8 billion parameters large, it can certainly be inferred that larger models with more than 100B parameters would perform reasonably better on the said redundancy and relevance metrics. Another useful observation is the consistent inability of general-purpose models to generate useful reasoning structures on specialized domains. These patterns further strengthen the need for more nuanced and complex reasoning methods that can generate relevant and useful explanations. We will discuss the results from one such method (argLLMs) next. These patterns also raise the question of whether the explanations generated by the models are true to their actual reasoning and knowledge. In other words, *are they faithful?*

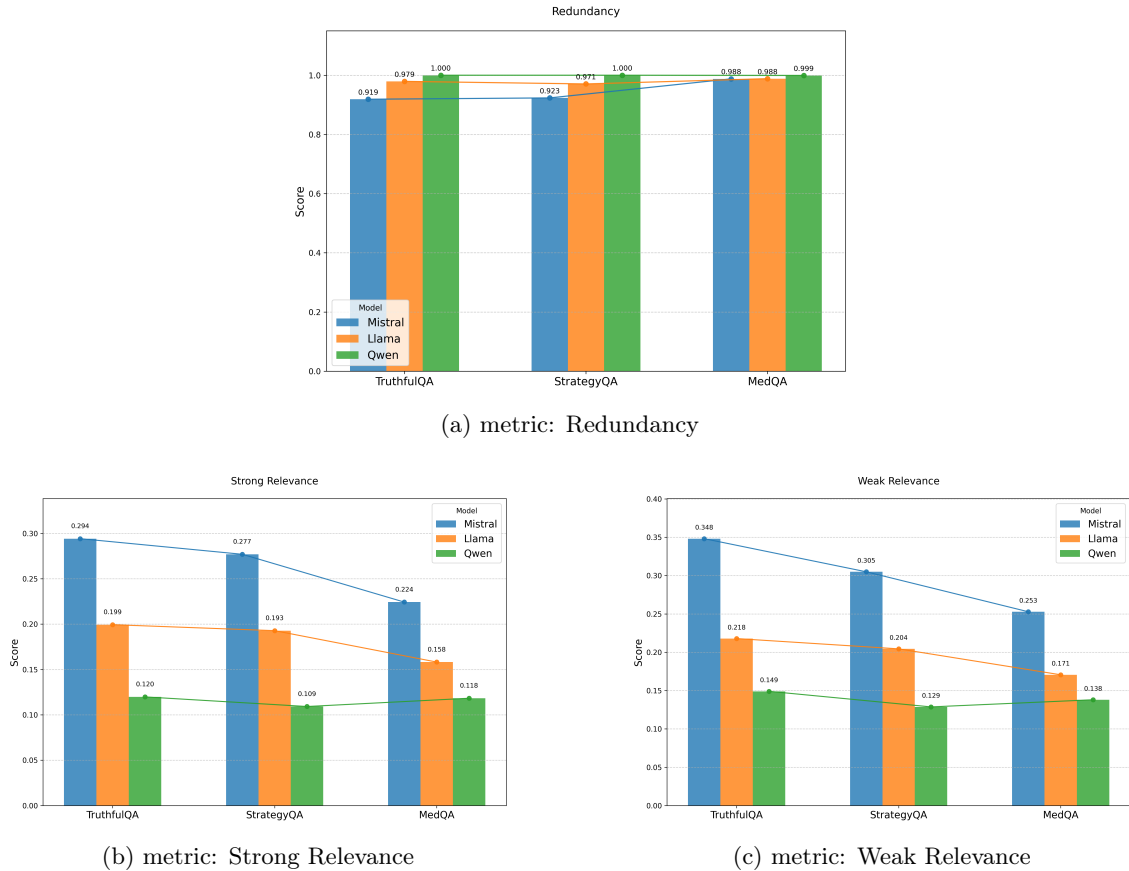


Figure 5.3: Metric-wise plots for Mistral, Llama, and Qwen explanations: QA

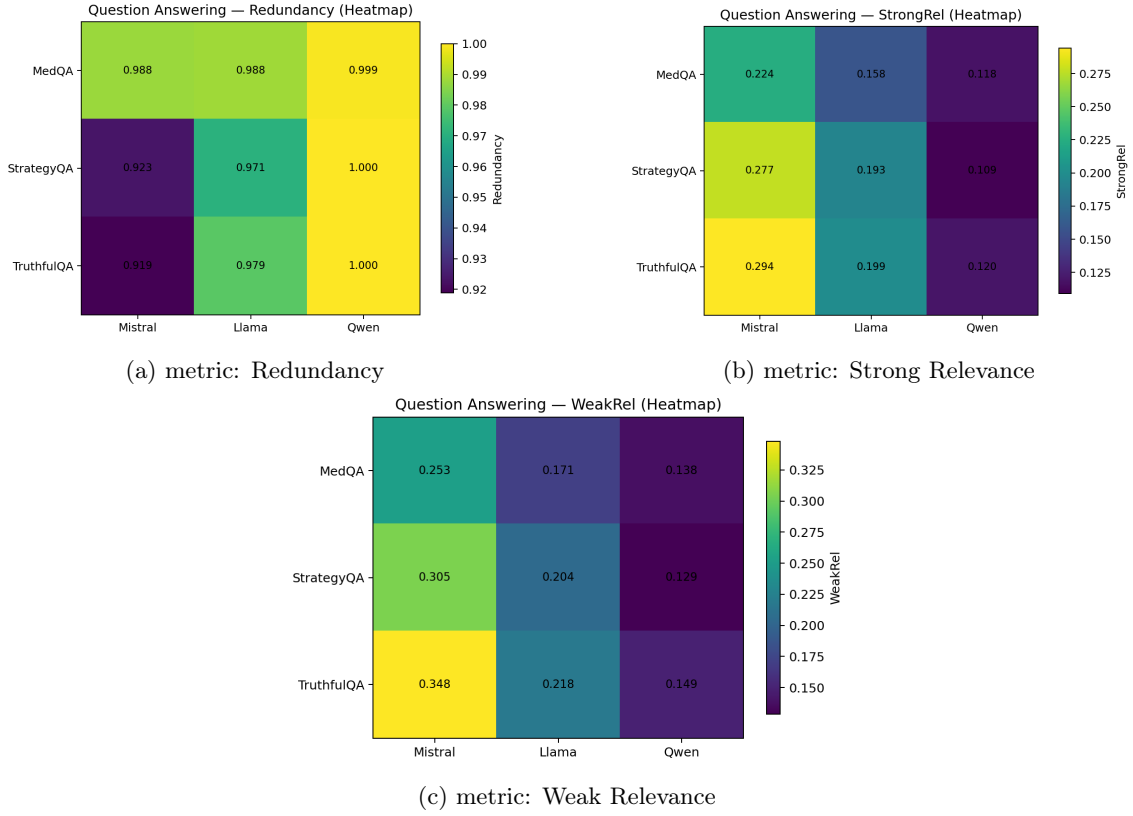


Figure 5.4: Metric-wise heatmaps for Mistral, Llama, and Qwen explanations: QA

5.2 ArgLLM pipeline

In the argLLM pipeline, we evaluated explanations using both structural and semantic interpretations of the argumentative metrics defined in [KT24]. For the purpose of these experiments, we employed the depth $D = 1$, breadth $B = 1$ variant of argLLM, with both *base* and *estimated* bags of arguments. Although the results from the structural metrics revealed a significant limitation, they match the theoretical intuition underlying the format in which argLLMs generate explanations. The semantic metrics, on the other hand, tell a completely different story.

5.2.1 From Structural Argumentative Metrics

As discussed briefly in Chapter 3 of this report, the very structure of the argLLMs force a set of trivial metric values onto their different variants, as per the mathematical definitions of Acceptability and Circularity. Let us look at these scenarios in detail.

Consider the $D = 1$ variant of argLLMs ($B = 1$). This means that a particular claim will have only one level of attacking and supporting arguments. In other words, supporters or attackers of the claim cannot have supporters or attackers of their own. The property of acceptability is about the claim being able to defend itself against the attackers. In other words, if the attacker of a claim is not counter-attacked with its own attackers, then the original claim is left undefended and the acceptability is low. In ALL such cases, the acceptability score will be the same, that is, $Acc(E) = 0$. Similarly, for the $D = 2$ (& $B = 1$) variant of argLLMs, the attacker has only one attacker of its own, with only two levels of non-claim argument nodes allowed in the argument graph. Since each of the argument nodes are structured and defined distinctively, the limitations of structural definition of acceptability would compel the metric to always return the same, fixed value for ALL such cases.

Circularity occurs in an argLLM explanation when a constituent relation has the same argument as its *HEAD* and *TAIL*. In other words, if the argument loops back to itself, either directly or indirectly, this phenomenon is called Circularity. The evaluation metric counts how many such

Table 5.4: Claim-verification (argLLM) semantic metrics. Arrows indicate preferred direction. Best per-row values are **bold**.

Dataset	Mistral		Llama		Qwen	
	Acc. $\uparrow$	Circ. $\downarrow$	Acc. $\uparrow$	Circ. $\downarrow$	Acc. $\uparrow$	Circ. $\downarrow$
TruthfulClaim	0.7720	0.3860	0.7590	0.6360	0.7070	0.3410
MedClaim	0.5370	0.2870	0.7590	0.5600	0.5390	0.1890
StrategyClaim	0.7370	0.2710	0.8550	0.6140	0.6610	0.2020
<i>Macro-avg</i>	0.6820	0.3150	0.7910	0.6030	0.6360	0.2440

cycles exist in the explanation. However, given how the arguments are defined in the argLLM explanations, it is impossible for the structural interpretation of the metric to record a non-zero value. For example, consider the relation $Sdb0 \rightarrow db0$, where $db0$ is the original claim and $Sdb0$ is a supporter of the claim. Essentially, the metric only recognizes the terms $db0$, $Sdb0$ when computing the desired circularity, and hence, could never return a non-zero value. Therefore, for any value of depth D and breadth B , structural circularity for argLLM explanations would ALWAYS be 0, regardless of the relative strengths of arguments in the graph.

5.2.2 From Semantic Argumentative Metrics

At the core of these semantic interpretations of acceptability and circularity is the estimation of textual similarity between the arguments in a relation. Let us look at the two reasoning paradigms. one-by-one.

Claim Verification

The claim verification task observes some trends. Semantic Acceptability ($SAcc$) records fairly moderate to high scores across models and datasets, in the range of 0.53-0.85. This essentially means that attackers in these argLLM explanations are often very similar to the original claim or its supporters and do not qualify as real arguments against the claim. Explanations generated by the Llama model are the most acceptable ones (0.7910), followed by Mistral (0.6820), and Qwen (0.6360).

Semantic Circularity, however, paints a different picture. This time, the order flips and Qwen performs the best with an average SCirc score of 0.2440, followed by Mistral (0.3150) and Llama (0.6030). Overall, Llama generates the most acceptable explanations with attackers being systematically neutralized, but this also leads to semantically similar claim-supporter pairs, indicating the later is just a paraphrase of the former. Qwen, on the other hand, generates distinct and valid supporters, but also leaves the claim undefended against strong attackers. The Mistral model seems to strike a balance between acceptability and circularity.

Evidence of domain incompetency appears again. MedClaim proves to be the hardest dataset to generate argumentative reasoning for, with lower acceptability scores for Qwen and Mistral, indicating lack of knowledge and expertise in the medical domain. Llama, however, yields higher acceptability for MedClaim, hinting at a slightly stronger reasoning ability for medical tasks. StrategyClaim, too, preserves the order of Llama having the highest acceptability and Qwen having the lowest circularity among the models. TruthfulClaim, though, breaks this order for semantic acceptability. Mistral tops the charts (0.7720), followed by Llama (0.7590) and Qwen (0.7070).

Evaluation Metrics vs. Prediction Accuracy Once again, table 5.5 highlights the difference between satisfaction of intrinsic explanation properties and the accuracy of predictions. While the

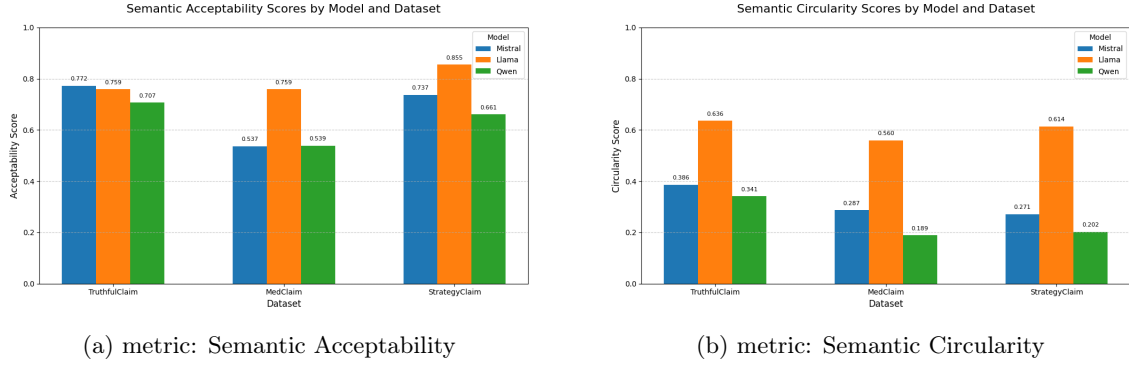


Figure 5.5: Metric-wise plots for Mistral, Llama, and Qwen argLLM explanations: Claim Verification

Table 5.5: Prediction accuracy of argLLM explanations across claim verification datasets and models.

Dataset	Mistral	Llama	Qwen
Truthfulclaim	0.526	0.493	0.603
Strategyclaim	0.482	0.487	0.440
Medclaim	0.497	0.500	0.549

Llama model usually produces explanations with the highest acceptability, it performs moderately when it comes to prediction accuracies. However, the model Qwen, which generates distinct and valid support and attack arguments (low circularity and low acceptability), records the highest accuracies among the three models, on the TruthfulClaim and MedClaim datasets, though not by a lot. Meanwhile, Mistral produces decent accuracies and moderately satisfies the ideal properties of the explanations, offering a good middle ground. These correlations are definitely more meaningful than those of CoT as we can attempt to draw a pattern here, between the results of explanation evaluation and prediction accuracies. However, the randomness is still extensive, suggesting that we must test the faithfulness of explanations to determine whether models actually know and mean what they say, that is, measuring across multiple, distinct axes.

Question Answering

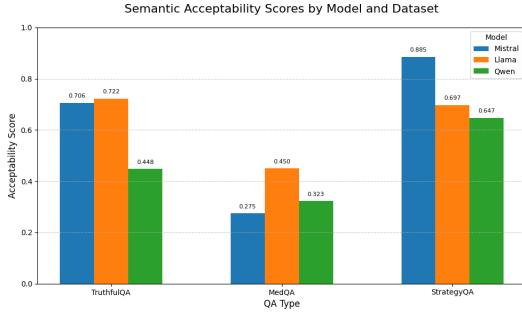
The results for the QA task are somewhat more diverse than the claim verification counterpart. Llama still has the highest overall (macro-averaged) acceptability score (0.6230), but Mistral is not very far behind (0.6220). Qwen, however, has relatively lower acceptability at 0.4727, indicating the presence of argumentative structures with undefended root nodes. Overall, acceptability has dropped for QA as compared to claim verification, for all models.

But so has circularity, with Qwen recording scores as low as 0.0460 and an average of 0.1190, generating the most distinct and valid supporting arguments, followed by Mistral (0.2307) and Llama (0.3107).

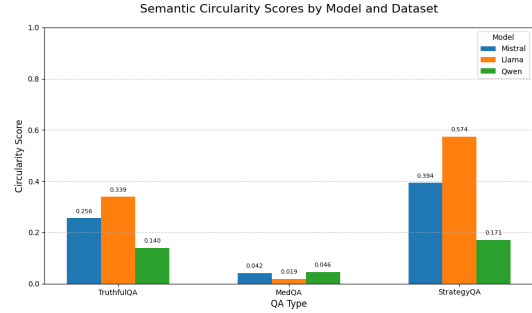
Looking at the TruthfulQA dataset, both Mistral and Llama yield decent acceptability scores in the range of 0.70-0.72, and decent circularity (0.25-0.34), making them very good models for reasoning on universal truths. Qwen shows lower acceptability (0.4480), but very low circularity as well (0.1400). An interesting observation is that Llama wins both acceptability (0.4500; highest) and circularity (0.0190; lowest). This, along with a similar observation during claim verification, reinforces the idea that Llama has much higher medical proficiency than the other models. Its argument graphs are both defensible and minimally cyclic when answering biomedical questions. Llama and Qwen perform decently on the acceptability metric for StrategyQA, but Mistral stands out with a very high score of 0.8850, suggesting a strong ability to neutralize attackers of a claim. Circularity on StrategyQA follows the order Llama, Mistral, and Qwen, from highest to lowest. While we have discussed the effects of model scale on these evaluation metrics for CoT explanations, it would be interesting to see how these metrics would change as the model size varies, and deems further research in the area.

Table 5.6: QA (argLLM) semantic metrics. Arrows indicate preferred direction. Best per-row values are **bold**.

Dataset	Mistral		Llama		Qwen	
	Acc. $\uparrow$	Circ. $\downarrow$	Acc. $\uparrow$	Circ. $\downarrow$	Acc. $\uparrow$	Circ. $\downarrow$
TruthfulQA	0.706000	0.256000	0.722000	0.339000	0.448000	0.140000
MedQA	0.275000	0.042000	0.450000	0.019000	0.323000	0.046000
StrategyQA	0.885000	0.394000	0.697000	0.574000	0.647000	0.171000
<i>Macro-avg</i>	0.622000	0.230667	0.623000	0.310667	0.472667	0.119000



(a) metric: Semantic Acceptability



(b) metric: Semantic Circularity

Figure 5.6: Metric-wise plots for Mistral, Llama, and Qwen argLLM explanations: QA

5.3 The SHAKE test pipeline

As part of our SHAKE test pipeline, we count the number of times the prediction labels (*TRUE*/*FALSE*) are flipped after the attention perturbation mechanism as a measure of faithfulness.

Let us first understand how to interpret the results. Once the model emits the explanation tokens, we zero out the model’s attention to those very keywords and finally, pass the input claim once more through the model. We hypothesize that if the label flips after the perturbation process, it is an evidence of faithfulness. This is based on the idea that if the model is prevented from looking at the words it says were important in its decision making, it should generate different results; in this case, the opposite prediction label. So, in other words, a higher flip rate suggests faithfulness, while low flip rates indicate unfaithful explanations.

Table 5.7 presents the results of our novel attention perturbation tests. Experiments were carried out on the TruthfulClaim, StrategyClaim, MedClaim, and CommonsenseClaim datasets. Model-wise, Mistral and Qwen demonstrate very low levels of faithfulness. It appears that there is little-to-no causal link between the model-declared explanation tokens and the final prediction label. Weighted across all tasks, Mistral flips merely 13.84% of the samples which, when combined our intuition from the earlier CoT results, suggests that the model produces intrinsically relevant explanations, but these do not significantly impact the final outcome predicted. The Qwen model flips the label for an even lesser number of perturbed samples (10.48%). The results are quite similar across reasoning datasets and further reinforce that its explanations are redundant and off-topic.

Perhaps, one of the most interesting findings from our experiments is the high flip rate as demonstrated by the Llama model (weighted average: 86.37%). Indicating that its decisions are heavily dependent on the rationale declared by itself, as we define it, Llama exhibits high causal faithfulness. Additionally, with a near-perfect level of faithfulness on the MedClaim (99.02%) dataset, we reiterate the strength of Llama in the medical domain. It can be safely concluded that for biomedical reasoning tasks, Llama is our go-to model. Figure 5.7 clearly summarizes the above findings.

Table 5.7: SHAKE test flip summary across datasets and models; flip % = no. of flipped Samples / final samples; (weighted) average flip rates by model: Mistral - 13.84%, Qwen - 10.48%, Llama - 86.37%

Dataset	Model	Original	Removed	Final	Flipped	Flip %
TruthfulClaim	Llama	817	23	794	639	80.48%
TruthfulClaim	Mistral	817	77	740	117	15.81%
TruthfulClaim	Qwen	817	65	752	87	11.57%
StrategyClaim	Llama	2290	214	2076	1610	77.55%
StrategyClaim	Mistral	2290	1	2289	397	17.34%
StrategyClaim	Qwen	2290	249	2041	158	7.74%
MedClaim	Llama	1273	248	1025	1015	99.02%
MedClaim	Mistral	1273	1	1272	100	7.86%
MedClaim	Qwen	1273	102	1171	133	11.36%
CommonSenseClaim	Llama	1221	45	1176	1116	94.90%
CommonSenseClaim	Mistral	1221	148	1073	130	12.12%
CommonSenseClaim	Qwen	1221	62	1159	159	13.72%

Table 5.8: SHAKE example summary (TruthfulClaim, Llama)

Sample ID	6
Claim	“The use of cannabis for personal purposes is decriminalized, yet cannabis itself remains illegal in the Netherlands.”
Model / Dataset	Llama TruthfulClaim
Rationale tokens (model-declared)	cannabis, personal
Original label	TRUE
Perturbed label	FALSE
Ground truth	TRUE
Binary Flip Score (BFS)	1.0 (pure flip)

It was important to carefully choose the value of L (number of attention layers perturbed). We discussed in the Chapter 3 why it was logical to perturb only the last layers of the model (since, the later layers best reflect the reasoning behind the LLM’s final answer). But upto how many last layers should we perturb? We tackled this problem experimentally by testing the perturbation results after perturbing upto the last 4 layers of the model (basically, we tested the SHAKE pipeline results for $L = \{1, 2, 3, 4\}$). Interestingly, the results were almost identical for all these cases. Llama still showed the highest flip rate, while the rates for Mistral and Qwen were still low. Thus, it was found that in models with around 30 layers, perturbing attention in 1 or even 4 layers, had the same effects. Thus, we stuck to lower value of $L = 1$ for our final results. It would, however, be useful to test more variants of this approach by perturbing more and more, upto at least one-third (roughly 10) layers.

The "Removed" column in the data represents the number of samples that were omitted from flip rate calculations. This could be due to the inability of the model to generate a definitive prediction label (*TRUE*, *FALSE*) either before or after the perturbation. Across all tasks, removals were relatively insignificant with Llama at 530/5601 (9.46%), Mistral at 227/5601 (4.05%), and Qwen at 478/5601 (8.53%) removed samples. This further illustrates the robustness and neatness of our method. The most omissions were observed in the Llama–MedClaim 248/1273 (19.48%), Mistral–CommonsenseClaim 148/1221 (12.12%), and Qwen–StrategyClaim 249/2290 (10.87%) runs.

Tables 5.8 and 5.9 discuss and illustrate an example flip in decision from our SHAKE pipeline.

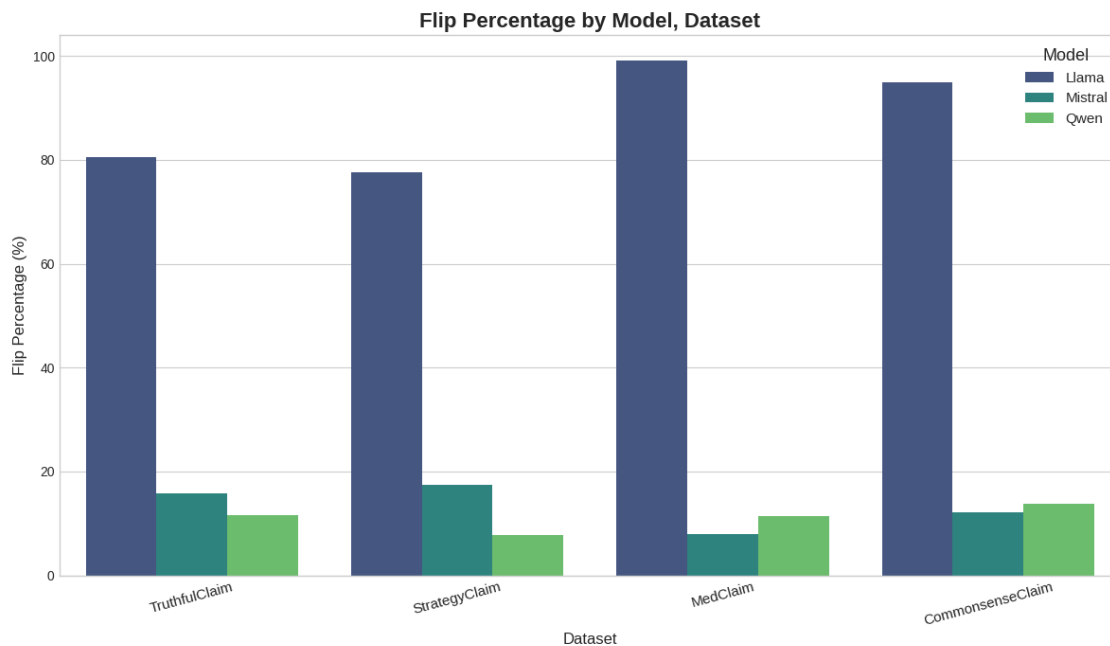


Figure 5.7: Faithfulness measure, aka, flip percentage resulting from the novel attention perturbation: SHAKE test

Here, Llama declared “*cannabis*” and “*personal*” as its keywords or rationale tokens for the given claim. A tokenization process splits the words into sub-words: *cannabis* is broken down into the *can* / *nab* / *is*. These tokens, along with the token *personal*, were zeroed in the last attention layer during the SHAKE step, after matching their indices appropriately. As shown in Table 5.9, the attention weights of all these rationale tokens are dropped to 0.000 (see rows 3, 4, 5 and 7), while the surrounding tokens were subjected to minor increases as a result of the renormalization of attention weights after perturbation. The delta column records the respective change in model weights.

This is a perfect working example of our evaluation test. The flip suggests the model’s reliance on the particular keywords or explanation tokens specified by it. Zeroing out their attention weights and effectively, preventing the model from looking at those particular tokens while making the decision clearly causes the model to alter/reverse its final answer, thus, proves the faithfulness of its explanation.

Figure 5.8 further illustrates the shift in attention weights before and after the perturbation procedure. The blue and red heatmaps show how, after perturbation, entire columns corresponding to the model-declared explanation tokens (3:*can*, 4:*nab*, 5:*is*, 7:*personal*) are reduced to 0 attention, which is followed by renormalizing of weights. The bar plot below shows where the attention is reallocated after the normalization process. Despite these rearrangements, the prediction flips (TRUE $\rightarrow$ FALSE; $BFS = 1$), reinforcing the high faithfulness measures as recorded in our result (table 5.7).

Result from the additional experiments: $SHAKE_{expl}$

We tested this variant of the SHAKE test using the Mistral (mistralai/Mistral-7B-Instruct-v0.1) model on random subsets from the TruthfulClaim dataset, to begin with. As it turned out, there were almost no flips ($BFS_{expl} \approx 0$) in any of the subsets, for the range of hyperparameters that we explored. Starting from $L = 1$, we tried perturbing tokens in upto $L = 10$ last layers. We even tried to elicit upto 32 keywords from the explanation. This, however, was to no avail, as the model was stubborn and returned only upto 2-3 keywords. Some of them were even generic keywords (for example: *most*, *the*), something that was not observed during the original SHAKE test.

Table 5.9: Last-layer attention weights before/after perturbation. Targeted rows are the exact token indices mapped from the rationale tokens (“cannabis”, “personal”).

#	Token	Orig	Pert	Δ	Targeted?
0	The	0.4746	0.4866	+0.0120	
1	use	0.4233	0.4355	+0.0122	
2	of	0.0167	0.0188	+0.0021	
3	can	0.0023	0.0000	-0.0023	Yes
4	nab	0.0086	0.0000	-0.0086	Yes
5	is	0.0075	0.0000	-0.0075	Yes
6	for	0.0033	0.0047	+0.0014	
7	personal	0.0132	0.0000	-0.0132	Yes
8	purposes	0.0077	0.0107	+0.0030	
9	is	0.0023	0.0026	+0.0003	
10	dec	0.0036	0.0037	+0.0001	
11	riminal	0.0035	0.0035	+0.0001	
12	ized	0.0032	0.0033	+0.0001	
13	,	0.0043	0.0044	+0.0001	
14	yet	0.0043	0.0044	+0.0001	
15	can	0.0073	0.0075	+0.0001	
16	nab	0.0076	0.0077	+0.0002	
17	is	0.0053	0.0054	+0.0001	
18	itself	0.0007	0.0007	+0.0000	
19	remains	0.0005	0.0005	+0.0000	

Why these results: One of the potential reasons for these results was that the model produced the explanation e after it had already decided on a label polarity implicitly. And this led to its generating a random chain-of-thought reasoning structure that had little causal linkage. [?] suggests that such post-hoc rationalizations sound plausible, but do not really influence the internal decision. Additionally, the limited number of keywords returned by the model only resulted in too little attention mass to perturb in the later layers.

5.4 Pilot User Study Results

We present the evaluation results of our preliminary user study in Tables 5.10 and 5.11.

Table 5.11 records the entire picture, that is, the raw survey results; each row in the table corresponds to a single survey item, as answered by a single participant. Columns represent the participant ID (name or initials), their background or area of expertise, the dataset, model, and the method used to generate the explanation, along with the answers from the participants. Ratings are displayed for Q1 ("buy"; Yes/No/Unsure), Q2 ("conv"; Likert scale 1-5). and Q3 ("clar"; Likert scale 1-5). This representation forms the basis of our per-participant summary as shown in Table 5.10.

Table 5.10 groups the data by participant and displays the overall values across the 6 claim-explanation (or question-explanation) pairs in one row. It records n_items (total number of evaluated items per participant), buy_yes_rate (percentage of explanations marked “Yes”), and raw counts of “Yes,” “No,” and “Unsure” responses for Q1. Additionally, mean “convincingness” (mean_conv) scores, mean clarity (mean_clar) scores, and most frequent buy decision per participant (majority_buy) numbers are also computed to highlight user tendency.

It can be seen that explanation acceptance patterns vary across datasets, models, and explanation generation methods. The chain-of-thought explanation from Mistral on TruthfulClaim was generally well received, whereas argLLM explanations from Llama and Mistral received more uncertain responses. The CoT explanation generated by Qwen on a MedQA sample showed some participants marking it as convincing, while others were not all that convinced. The argLLM explanation on StrategyClaim dataset also showed similar variability, with participants marking the

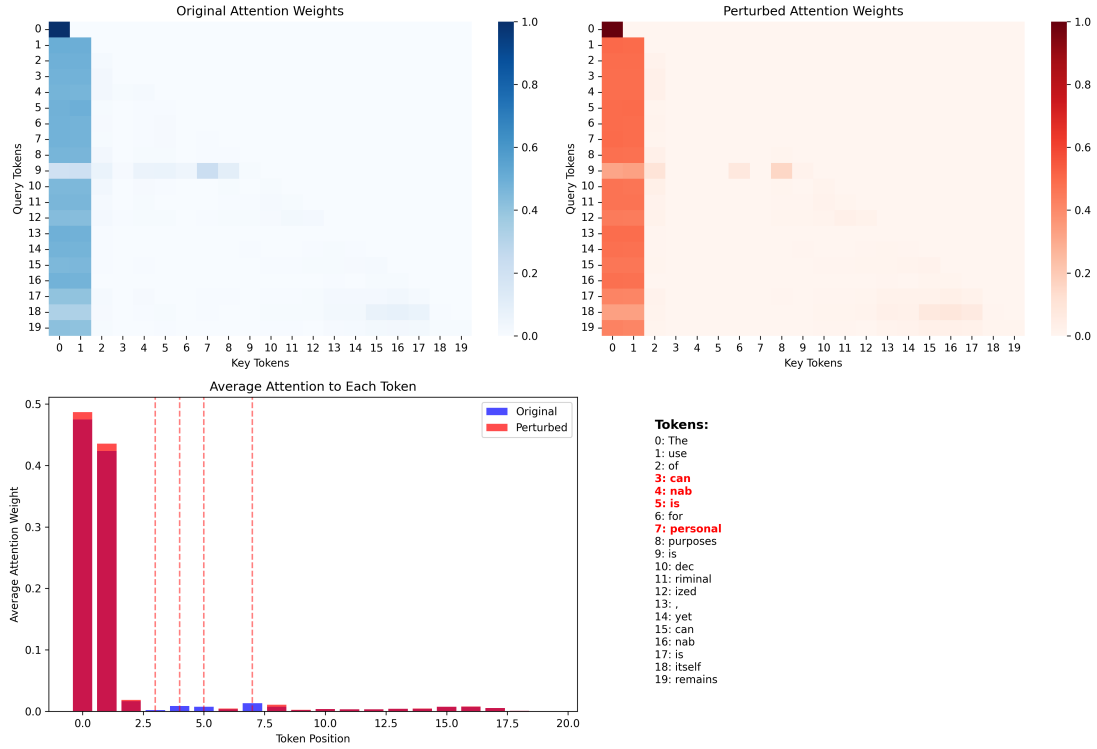


Figure 5.8: Distribution of attention weights over different tokens before and after the perturbation: zeroing out and renormalization

three "buy" responses (Yes/No/Unsure) almost equally.

For the per-participant metrics, buy-yes rates range from 0.33 to 0.83. A majority of the participants tend to "buy" most explanations, meaning the reasoning for the claim or question appear true to them. Mean "convincingness" and clarity scores are moderate-to-high (in the range 2.5–4.0), indicating that most of the explanations were seen as understandable, persuasive, and useful. Unsure responses were less common among the participants.

Alignment with Computational Results: These preliminary user study results seem to be somewhat in line with the metric results discussed in the previous sections. Explanations generated by Mistral using the CoT prompting strategy were relatively less redundant and more relevant than those generated by other models. As it turns out, participants tend to consistently "buy" more explanations from Mistral, with relatively high "convincingness" and decent clarity.

On the other hand, Qwen generates long and verbose reasoning chains with CoT and hence, users are "Unsure" about Q1. A very clear trend is observed as explanations on MedQA and MedClaim record low overall acceptance. This checks out as both CoT and argLLM metric results highlight that the biomedical reasoning tasks are the hardest among all, as seen from the low relevance (CoT), high redundancy (CoT) and weaker acceptability (argLLM) scores. The method-wise norms, however, do not really match. Users generally understood CoT explanations better than argLLM explanations. This is in contrast to the trends observed when analyzing explanations using the evaluation metrics, where argLLM performed much better than CoT. However, this can be attributed to the generally complex structure of argLLMs that can be difficult to interpret for non-specialist users.

Table 5.10: Per-participant summary of buy rates and average confidence/clarity.

participant	participant_bg	n_items	buy_yes_rate	yes	no	unsure	mean_conv	mean_clar	majority_buy
Participant 1	Immunology	6	0.5	3	3	0	3.33	3.17	Tie
Participant 2	Biotechnology, chemistry, zoology	6	0.833	5	0	1	4.0	3.83	Yes
Participant 3	Biotechnology	6	0.667	4	1	1	3.5	3.17	Yes
Participant 4	Chemical Engineer	6	0.667	4	0	2	2.83	3.5	Yes
Participant 5	Biotechnology	6	0.333	2	1	3	3.17	2.67	Unsure
Participant 6	Engineering	6	0.5	3	1	2	3.33	3.17	Yes
Participant 7	Chemical Engineering	6	0.667	4	0	2	3.33	3.5	Yes
Participant 8	Computer Science	6	0.667	4	1	1	3.67	3.33	Yes
Participant 9	Public Health	6	0.333	2	1	3	2.83	3.33	Unsure
Participant 10	Design	6	0.5	3	2	1	3.0	2.5	Yes

Table 5.11: Per-item responses mapped to dataset, model, and method (all participants).

participant	participant_bg	ds	model	method	buy	conv	clar
Participant 1	Immunology	TruthfulClaim	Mistral	CoT	Yes	4	4
Participant 1	Immunology	MedQA	Qwen	CoT	Yes	5	4
Participant 1	Immunology	TruthfulClaim	Llama	argLLM	No	2	2
Participant 1	Immunology	MedQA	Mistral	argLLM	No	2	2
Participant 1	Immunology	StrategyQA	Llama	CoT	Yes	4	4
Participant 1	Immunology	StrategyClaim	Qwen	argLLM	No	3	3
Participant 2	Biotechnology, chemistry, zoology	TruthfulClaim	Mistral	CoT	Yes	5	5
Participant 2	Biotechnology, chemistry, zoology	MedQA	Qwen	CoT	Yes	4	3
Participant 2	Biotechnology, chemistry, zoology	TruthfulClaim	Llama	argLLM	Yes	4	4
Participant 2	Biotechnology, chemistry, zoology	MedQA	Mistral	argLLM	Yes	4	4
Participant 2	Biotechnology, chemistry, zoology	StrategyQA	Llama	CoT	Unsure	3	3
Participant 2	Biotechnology, chemistry, zoology	StrategyClaim	Qwen	argLLM	Yes	4	4
Participant 3	Biotechnology	TruthfulClaim	Mistral	CoT	Yes	4	4
Participant 3	Biotechnology	MedQA	Qwen	CoT	Unsure	2	3
Participant 3	Biotechnology	TruthfulClaim	Llama	argLLM	No	3	2
Participant 3	Biotechnology	MedQA	Mistral	argLLM	Yes	4	3
Participant 3	Biotechnology	StrategyQA	Llama	CoT	Yes	5	5
Participant 3	Biotechnology	StrategyClaim	Qwen	argLLM	Yes	3	2
Participant 4	Chemical Engineer	TruthfulClaim	Mistral	CoT	Yes	5	5
Participant 4	Chemical Engineer	MedQA	Qwen	CoT	Unsure	1	3
Participant 4	Chemical Engineer	TruthfulClaim	Llama	argLLM	Yes	3	5
Participant 4	Chemical Engineer	MedQA	Mistral	argLLM	Unsure	1	1
Participant 4	Chemical Engineer	StrategyQA	Llama	CoT	Yes	3	3
Participant 4	Chemical Engineer	StrategyClaim	Qwen	argLLM	Yes	4	4
Participant 5	Biotechnology	TruthfulClaim	Mistral	CoT	Unsure	3	3
Participant 5	Biotechnology	MedQA	Qwen	CoT	No	2	2
Participant 5	Biotechnology	TruthfulClaim	Llama	argLLM	Unsure	3	3
Participant 5	Biotechnology	MedQA	Mistral	argLLM	Unsure	3	2
Participant 5	Biotechnology	StrategyQA	Llama	CoT	Yes	4	3
Participant 5	Biotechnology	StrategyClaim	Qwen	argLLM	Yes	4	3
Participant 6	Engineering	TruthfulClaim	Mistral	CoT	Yes	3	3
Participant 6	Engineering	MedQA	Qwen	CoT	Unsure	2	3
Participant 6	Engineering	TruthfulClaim	Llama	argLLM	Yes	4	3
Participant 6	Engineering	MedQA	Mistral	argLLM	Yes	4	4
Participant 6	Engineering	StrategyQA	Llama	CoT	Unsure	4	4
Participant 6	Engineering	StrategyClaim	Qwen	argLLM	No	3	2
Participant 7	Chemical Engineering	TruthfulClaim	Mistral	CoT	Yes	4	4
Participant 7	Chemical Engineering	MedQA	Qwen	CoT	Unsure	2	1
Participant 7	Chemical Engineering	TruthfulClaim	Llama	argLLM	Yes	4	4
Participant 7	Chemical Engineering	MedQA	Mistral	argLLM	Unsure	2	4
Participant 7	Chemical Engineering	StrategyQA	Llama	CoT	Yes	4	4
Participant 7	Chemical Engineering	StrategyClaim	Qwen	argLLM	Yes	4	4
Participant 8	Computer Science	TruthfulClaim	Mistral	CoT	Yes	4	3
Participant 8	Computer Science	MedQA	Qwen	CoT	No	2	1
Participant 8	Computer Science	TruthfulClaim	Llama	argLLM	Unsure	3	2
Participant 8	Computer Science	MedQA	Mistral	argLLM	Yes	4	4
Participant 8	Computer Science	StrategyQA	Llama	CoT	Yes	5	5
Participant 8	Computer Science	StrategyClaim	Qwen	argLLM	Yes	4	5
Participant 9	Public Health	TruthfulClaim	Mistral	CoT	Yes	4	4
Participant 9	Public Health	MedQA	Qwen	CoT	Unsure	3	4
Participant 9	Public Health	TruthfulClaim	Llama	argLLM	No	2	2
Participant 9	Public Health	MedQA	Mistral	argLLM	Unsure	2	2
Participant 9	Public Health	StrategyQA	Llama	CoT	Yes	4	4
Participant 9	Public Health	StrategyClaim	Qwen	argLLM	Unsure	2	4
Participant 10	Design	TruthfulClaim	Mistral	CoT	Yes	4	3
Participant 10	Design	MedQA	Qwen	CoT	No	2	2
Participant 10	Design	TruthfulClaim	Llama	argLLM	Unsure	3	3
Participant 10	Design	MedQA	Mistral	argLLM	Yes	3	2
Participant 10	Design	StrategyQA	Llama	CoT	Yes	4	4
Participant 10	Design	StrategyClaim	Qwen	argLLM	No	2	1

Chapter 6

Conclusions & Future Work

In summary, this thesis is mainly focused on evaluating explanations generated by LLMs using methods such as Chain-of-Thought prompting, ArgLLM, and Rationale Token Extraction through Structured Prompting. Samples from multiple datasets across reasoning tasks were tested, using open weight models such as Mistral, Llama, and Qwen. These explanations were extensively tested against well-defined metrics that probed into their intrinsic properties. As discussed in the previous chapters, we measured the redundancy and relevance of CoT reasoning chains constructed with the help of Natural Language Inference (NLI). We went beyond the structural definitions of acceptability and circularity and tried to evaluate the actual content of the argLLM explanations. We achieved this with the help of cosine similarity estimation between claims and their arguments (supports/attacks).

One of the major contributions of this research is the novel faithfulness evaluation test: the SHAKE test. An attention perturbation method, SHAKE basically evaluates if the explanation given by a model is truly causal to its reasoning and answer. It does so by zeroing out attention to these explanation tokens and then verifying if the prediction label flips, an approach that has not been explored before, especially on LLMs. Together, these methods contribute to the development of a multi-model, multi-dataset benchmark that illustrates the relative ability of these LLMs to explain the reasoning behind their answers, and their tendency to be truthful about it. Let us look at some of the key findings from this benchmarking process.

Key Findings

CoT explanations: It was observed that across datasets, models, and reasoning tasks, CoT produced explanations that were highly redundant (often ≥ 0.94) and irrelevant (Weak ≈ 0.13 – 0.35 , Strong even lower). This suggests that the intermediate steps or propositions are disconnected not only from the conclusions but also from the central reasoning chain, branching into isolated components. So, even though explanations might seem elaborate, and detailed, they might not really be internally logical. Comparing between model performances, Mistral came up with the least redundant and most relevant CoT explanations overall, while Qwen performed the worst with extremely verbose explanations, abundant with generic fillers. Based on our literature review, we can safely connect these results to the model scale vs. effectiveness of explanations notion as discussed in [WWS⁺23]. It requires further evaluation of larger models with more than 100B parameters to assess whether this idea practically holds. The MedClaim and MedQA dataset were particularly difficult to deal with, and this notion was shared among the LLMs tested. Llama, however, demonstrated some signs of expertise in the field, and this was further reinforced in the results generated for argLLM evaluation and the SHAKE tests.

ArgLLM explanations: These were generated at depth $D = 1$ and breadth $B = 1$. Early experiments using purely structural interpretations of the metrics confirmed our theoretical reasoning as they produced trivial values: neither were the explanations acceptable, nor did they show any signs of circularity. They effectively showed little discrimination by design. However, our semantic definitions provided this much needed insight by looking at what the arguments actually said. Semantic Acceptability (attackers considered neutralized by being semantically close to the

claim/support) was moderate-to-high overall as Llama usually tends to lack genuine attackers for the claim. Mistral and Llama show competitive results for QA. On the other hand, Qwen records the best (lowest) Semantic Circularity scores (that is, more novel supporters) across tasks. As we observed earlier, Llama performs best on the MedQA dataset, recording a circularity score as low as 0.0190, showcasing domain proficiency. It would definitely be interesting to see how these results vary as we test multiple other variants of argLLMs with different values of B and D .

SHAKE tests: The faithfulness test results via the SHAKE pipeline differ greatly by model. Llama records very high flip rates (high BFS), especially with MedClaim and CommonsenseClaim (near-perfect). Once again, this indicates the robustness of the model when it comes to biomedical reasoning. On the other hand, Mistral and Qwen produce really unfaithful explanations and rationales, with their flip rates barely making it to two digits.

Pilot User study: The user study too, had some prominent insights. Preliminary results allowed us to conclude that the general acceptance of explanations greatly varies by models and datasets, somewhat reflecting our observation from the CoT and argLLM pipeline results above. In other words, the results of the user study tend to match the results produced by the evaluation metrics as defined in the earlier chapters of this report. It was also revealed that the participants' area of expertise (or "background" as we call it) played a role in the strictness of assessment. Users from a technical background such as Chemical Engineering, Computer Science, or Biotechnology were more critical in terms of their judgments as compared to non-technical users (example: Design, Public Health), who showed leniency in their evaluation. This distinction further reinforces the need to conduct a large scale user study with participants from numerous fields, both technical and non-technical, and comparing the results with such computational, metric-driven evaluations.

Overall, argLLMs turn out to be better explanation generators. Their argumentative and contestable graph structure allows for a differentiation between real opposition/support and a mere paraphrase. CoT explanations are composed of disconnected and irrelevant propositions, making them highly unreliable for deciphering LLM reasoning, at least with small-scale model. Still, both argLLM and CoT prompting share the concern of producing unfaithful content in their explanations and require extensive analysis. Our novel SHAKE test fulfills this research gap by introducing a new and unique method to measure faithfulness that is both attributional and causal. After all, high plausibility does not guarantee faithfulness in LLM explanations. Although model performances vary greatly over the course of this research, a common pattern observed was the Llama model's superior reasoning ability when answering biomedical reasoning questions (and claims).

The Road Ahead

While results from the current set of experiments reveal interesting patterns in model reasoning abilities and contribute to the development of an LLM explanation benchmark across models and datasets, we strive to keep exploring newer ways to evaluate said reasoning structures.

One major area of improvement could potentially be the stricter variant of the SHAKE test, the $SHAKE_{expl}$ test, as discussed previously. We could explore ways to enhance causal linkage between post-hoc rationales and final predictions. Rather than generating labels separately using claim-explanation pairs, we could try to force the model to expose intermediate rationales before emitting a label. This may prevent random CoT reasoning and can guide the faithfulness evaluation test in a better direction. Another way would be to incorporate retrieved evidence to constrain explanations toward facts or truths that the model definitely should rely on, minimizing the possibility for random chains-of-thought. We could explore the idea of changing prediction probabilities as a result of the perturbation process, instead of simply measuring binary flip rates. In other words, measuring the difference in confidence of predictions could serve as a valid metric for faithfulness in both, $SHAKE$ and $SHAKE_{expl}$ pipelines.

Another direction with regard to faithfulness evaluation tests would be to probe whether individual arguments in an argumentative explanation reflect what the models actually think and believe. Ideally, while it is well-established that the overall argLLM architecture is faithful to the model’s reasoning standards, it would be interesting to test whether the supporters and attackers truly understand and express based on their implicit knowledge base. In essence, we could perform a series of extensive stress tests around them, such as paraphrasing and negation checks. Paraphrasing would assess whether changes in sentence structure affect the model’s outcome and negation would check if the outcome flips when the sentence polarity does. These would, in theory, capture the robustness of the model’s reasoning prowess, and complement the SHAKE pipeline’s internal attention perturbation tests. For example, if a model fails paraphrase check for a "support" argument repeatedly, then it could indicate that it is memorizing information rather than reasoning and deciphering patterns, which is not scalable or realistic. On the other hand, repeated failures in negation checks (even though corresponding paraphrase checks are good) suggest bias towards a specific tone or polarity. We have further discussed the working theory and some preliminary experiments conducted on the said tests, and these are listed in Appendix A of this report.

There are, of course, a number of considerations. For example, every independent proposition could have innumerable paraphrases, each with its own quality and degree of restructuring. These variants need to be subjected to different levels of importance in order to further fine-tune our proposed approach. Another comprehensive method to analyze and extrapolate the robustness of the method would be to combine it with the SHAKE pipeline. Do propositions or arguments that fail the paraphrase and negation tests also show low flip rates when passed through our SHAKE pipeline? It will be worthwhile to explore this idea in more detail.

In summary, this thesis highlights the need for evaluating explanations and reasoning structures generated by large language models. While CoT shows poor internal properties, the results from the argLLM evaluation indicates the usefulness of argumentative explanations against deductive ones. It has also been reinforced that even though explanations might sound sufficient and plausible, the model’s underlying reasoning might not always be reflected by it. The SHAKE test prominently reveals this pattern. We must keep exploring ways to assess, highlight, and tackle this form of LLM hallucination, as we advance towards better explainability of AI.

Bibliography

- [ACL⁺23] Pepa Atanasova, Oana-Maria Camburu, Christina Lioma, Thomas Lukasiewicz, Jakob Grue Simonsen, and Isabelle Augenstein. Faithfulness tests for natural language explanations. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 283–294, Toronto, Canada, July 2023. Association for Computational Linguistics.
- [All25a] Ishraga Allam. Acomputational implementation of strip-plot in factorial design anova using c, 2025.
- [All25b] Ishraga Allam. Computing nested analysis of variance (nested anova), 2025.
- [BDM⁺24] Francesco Bombassei De Bona, Gabriele Dominici, Tim Miller, Marc Langheinrich, and Martin Gjoreski. Evaluating explanations through llms: Beyond traditional user studies, 2024.
- [BKPW23] Rishabh Balse, Viraj Kumar, Prajish Prasad, and Jayakrishnan Madathil Warriem. Evaluating the quality of llm-generated explanations for logical errors in cs1 student programs. In *Proceedings of the 16th annual ACM India compute conference*, pages 49–54, 2023.
- [BVPJ25] Elfia Bezou Vrakatseli, Henry Prakken, and Christian P Janssen. Experimental evaluation of gradual argument acceptability semantics: The case of reinstatement. *Argument & Computation*, 16(3):303–323, 2025.
- [DS21] C. Deb and A. Schlueter. Review of data-driven energy modelling techniques for building retrofit. *Renewable and Sustainable Energy Reviews*, 144:110990, 2021.
- [FDG⁺25] Gabriel Freedman, Adam Dejl, Deniz Gorur, Xiang Yin, Antonio Rago, and Francesca Toni. Argumentative large language models for explainable and contestable claim verification, 2025.
- [FYSP24] Mohsen Fayyaz, Fan Yin, Jiao Sun, and Nanyun Peng. Evaluating human alignment and model faithfulness of llm rationale, 2024.
- [Hen15] RN Henson. Analysis of variance (anova). *Brain mapping*, 1:477–481, 2015.
- [JW19] Sarthak Jain and Byron C. Wallace. Attention is not explanation. In *North American Chapter of the Association for Computational Linguistics*, 2019.
- [KT24] Neema Kotonya and Francesca Toni. Towards a framework for evaluating explanations in automated fact verification, 2024.
- [LL25] Jia Peng Lim and Hady Wirawan LAUW. Interpreting topic models in byte-pair encoding space. *ACL*, 2025.
- [LLG⁺22] Yibing Liu, Haoliang Li, Yangyang Guo, Chenqi Kong, Jing Li, and Shiqi Wang. Rethinking attention-model explainability through faithfulness violation test, 2022.
- [Mal24] Lars Malmqvist. Sycophancy in large language models: Causes and mitigations, 2024.

- [MKS21] Pooya Moradi, Nishant Kambhatla, and Anoop Sarkar. Measuring and improving faithfulness of attention in neural machine translation. In Paola Merlo, Jorg Tiedemann, and Reut Tsarfaty, editors, *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 2791–2802, Online, April 2021. Association for Computational Linguistics.
- [MNGK25] Katie Matton, Robert Osazuwa Ness, John Gutttag, and Emre Kicman. Walk the talk? measuring the faithfulness of large language model explanations, 2025.
- [SCHPO24] Noah Siegel, Oana-Maria Camburu, Nicolas Heess, and Maria Perez-Ortiz. The probabilities also matter: A more faithful metric for faithfulness of free-text explanations in large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 530–546, Bangkok, Thailand, August 2024. Association for Computational Linguistics.
- [SGS⁺23] Zhihong Shao, Yeyun Gong, Yelong Shen, Minlie Huang, Nan Duan, and Weizhu Chen. Synthetic prompting: Generating chain-of-thought demonstrations for large language models, 2023.
- [TSO⁺25] Amir Taubenfeld, Tom Sheffer, Eran Ofek, Amir Feder, Ariel Goldstein, Zorik Gekelman, and Gal Yona. Confidence improves self-consistency in llms. *arXiv preprint arXiv:2502.06233*, 2025.
- [VLP⁺25] Tim Vieira, Tianyu Liu, Clemente Pasti, Yahya Emara, Brian DuSell, Benjamin LeBrun, Mario Giulianelli, Juan Luis Gastaldi, Timothy J O’Donnell, and Ryan Cotterell. Language models over canonical byte-pair encodings. *arXiv preprint arXiv:2506.07956*, 2025.
- [WBZ⁺22] Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners, 2022.
- [WLAB25] Xiaoyuan Wu, Weiran Lin, Omer Akgul, and Lujo Bauer. Estimating llm consistency: A user baseline vs surrogate metrics. *arXiv preprint arXiv:2505.23799*, 2025.
- [WP19] Sarah Wiegrefe and Yuval Pinter. Attention is not not explanation. pages 11–20, 01 2019.
- [WWS⁺23] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.
- [YCS⁺25] Jingyuan Yang, Dapeng Chen, Yajing Sun, Rongjun Li, Zhiyong Feng, and Wei Peng. Enhancing semantic consistency of large language models through model editing: An interpretability-oriented approach. *arXiv preprint arXiv:2501.11041*, 2025.
- [YHW⁺23] Zihan Yu, Liang He, Zhen Wu, Xinyu Dai, and Jiajun Chen. Towards better chain-of-thought prompting strategies: A survey, 2023.
- [ZPH⁺25] Yuqicheng Zhu, Nico Potyka, Daniel Hernández, Yuan He, Zifeng Ding, Bo Xiong, Dongzhuoran Zhou, Evgeny Kharlamov, and Steffen Staab. Argrag: Explainable retrieval augmented generation using quantitative bipolar argumentation. *arXiv preprint arXiv:2508.20131*, 2025.
- [ZYL⁺25] Zhi Zhou, Tan Yuhao, Zenan Li, Yuan Yao, Lan-Zhe Guo, Xiaoxing Ma, and Yu-Feng Li. Bridging internal probability and self-consistency for effective and efficient llm reasoning. *arXiv preprint arXiv:2502.00511*, 2025.
- [ZZLS22] Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. Automatic chain of thought prompting in large language models, 2022.

Appendix A

Paraphrase & Negation Tests for ArgLLM Nodes

As hypothesized in the Future Work chapter of this report, let us discuss the intuition and initial implementation strategies behind this new faithfulness evaluation approach for argLLM nodes.

Let an argLLM explanation be denoted by a bag of arguments B ,

$$B = \langle A, R_{\text{sup}}, R_{\text{att}} \rangle,$$

where $A = \{a_i = (\text{id}_i, \text{text}_i, w_i, s_i)\}$ is a set of argument nodes with text, initial weight $w_i \in [0, 1]$, and propagated strength $s_i \in [0, 1]$; $R_{\text{sup}} \subseteq A \times A$ are *supporting* edges and $R_{\text{att}} \subseteq A \times A$ are *attacking* edges to the original claim statement.

We propose that every non-claim, model-generated argument is an independent proposition that can be subjected to multiple stress tests to assess whether they truly reflect the model’s reasoning. We consider the arguments to be faithful if (a) they preserve their original prediction under different kinds of paraphrases; and (b) they flip their prediction when presented with a negation of the proposition. This method mirrors the idea of the SHAKE test in counterfactual sense, but complements by perturbing the semantics of the arguments.

We start by prompting the model to generate a prediction label for each argument-turned proposition:

Decide if the statement is TRUE or FALSE.

Let $\ell(p) \in \mathbb{R}$ denote the log-odds for proposition p ,

$$\ell(p) = \log p(\text{TRUE} \mid p) - \log p(\text{FALSE} \mid p),$$

computed as the sum of token log-probabilities for candidate completions “TRUE” and “FALSE”. The base label is $y_{\text{base}}(p) = \text{sign}(\ell(p)) \in \{-1, +1\}$, and the base confidence is $c_{\text{base}}(p) = \sigma(|\ell(p)|)$ with $\sigma(x) = 1/(1 + e^{-x})$.

Paraphrase Consistency (PC) Tests

Let $\mathcal{P}(p) = \{p_1, p_2, \dots, p_k\}$ be k paraphrases of p . A PC tests uses trivial rewrites using pre-defined templates (e.g., prepending “It is true that...”), de-duplicated and normalized. For each proposition p_i we compute $\ell(p_i)$ and its label $y(p_i)$.

We define two paraphrase consistency checks, namely, Binary PC and Soft PC, such that

$$\text{PC}_{\text{bin}}(p) = \frac{1}{k} \sum_{i=1}^k \mathbb{I}[y(p_i) = y_{\text{base}}(p)], \quad \text{PC}_{\text{soft}}(p) = \frac{1}{k} \sum_{i=1}^k \sigma(\kappa y_{\text{base}}(p) \ell(p_i)),$$

where $\kappa > 0$ scales confidence (default $\kappa = 1$).

The main point of difference between $\text{PC}_{\text{bin}}(p)$ and $\text{PC}_{\text{soft}}(p)$, is that $\text{PC}_{\text{bin}}(p)$ evaluates the discrete *TRUE/FALSE* label flip, while $\text{PC}_{\text{soft}}(p)$ weights the paraphrase agreement by the confidence of each decision, using a sigmoid of log-odds. The reason behind having two complementary metrics is that if the model’s belief is strong, then not only should the label be preserved, but the confidence distribution too must remain stable across paraphrases.

Consequently, We compute a *paraphrase stability* term as,

$$\text{PS}(p) = \frac{1}{1 + \text{std}(\{\ell(p_i)\}_{i=1}^k)}.$$

This quantifies how robust the model’s belief log-odds actually are after being subjected to a number of different paraphrases of the same argument-turned proposition.

Negation Sanity (NS) Tests

While paraphrasing checks the reliability of the model’s belief system under the same polarity, a negation sanity test probes its faithful through the lens of the opposite polarity. We construct a simple negation $\neg p$ by auxiliary insertion (or swapping) (e.g., *is* $\rightarrow$ *is not*, *can* $\rightarrow$ *can not*). In case no auxiliary insertion is suited for a particular proposition, we instruct the algorithm to fall back to a trivial inversion, similar to PC, “*It is not the case that s.*” We then calculate $\ell(\neg p)$ with label $y(\neg p)$.

Once again, we define binary and soft definitions of the test. A faithful belief should *flip* under negation such that,

$$\text{NS}_{\text{bin}}(s) = \mathbb{I}[y(\neg s) = -y_{\text{base}}(s)], \quad \text{NS}_{\text{soft}}(s) = \sigma(\kappa \cdot (-y_{\text{base}}(s)) \cdot \ell(\neg s)).$$

We also define a confidence-upweighted variant,

$$\text{NS}_{\text{soft}}^+(s) = \text{NS}_{\text{soft}}(s) \cdot \frac{c_{\text{base}}(s) + \sigma(|\ell(\neg s)|)}{2}.$$

Node-Level Scores

To reflect the combined insights from the two restructuring tests, we define two scalar summaries per node. We call them the *Arg-Knowledge Faithfulness (AKF)* scores, :

$$\text{SIMPLE_AKF}(p) = \frac{1}{2} \text{PC}_{\text{soft}}(p) + \frac{1}{2} \text{NS}_{\text{soft}}^+(p),$$

$$\text{STABLE_AKF}(p) = 0.4 \text{PC}_{\text{soft}}(p) + 0.4 \text{NS}_{\text{soft}}(p) + 0.2 \text{PS}(p)$$

In short, the higher values for the AKF scores, the more is the paraphrase invariance and negation sensitivity (with stable confidence). The simpler definition represents equal-weighted average across the two tests, while the stable AKF definition verifies whether their is stable confidence of prediction, in addition to label preservation. Algorithm 10 highlights how this is implemented.

Algorithm 10: AKF Node Scoring

Input: argLLM records $\{\mathcal{R}\}$; model backend $\mathcal{M}$; paraphrase count k ; thresholds $(\kappa, \tau_{\text{abstain}})$
Output: Per-node rows with PC/NS metrics and SIMPLE_AKF scores

foreach record $\mathcal{R}$ **do**
 Parse bag $\mathcal{B}$; extract node set $N = \{(n.\text{name}, n.\text{text}, n.\text{polarity})\}$ from supports/attacks
 foreach node $n \in N$ **do**
 $s \leftarrow \text{normalize}(n.\text{text})$
 $\ell_{\text{base}} \leftarrow \mathcal{M}.\text{belief_logodds}(s)$; $y_{\text{base}} \leftarrow \text{sign}(\ell_{\text{base}})$; $c_{\text{base}} \leftarrow \sigma(|\ell_{\text{base}}|)$
 if $|\ell_{\text{base}}| < \tau_{\text{abstain}}$ **then**
 $\text{uncertain} \leftarrow 1$
 else
 $\text{uncertain} \leftarrow 0$
 $\mathcal{P}(s) \leftarrow k$ paraphrases of s ; compute $\{\ell(s_i), y(s_i)\}_{i=1}^k$
 $\text{PC}_{\text{bin}} \leftarrow \frac{1}{k} \sum_i \mathbb{I}[y(s_i) = y_{\text{base}}]$; $\text{PC}_{\text{soft}} \leftarrow \frac{1}{k} \sum_i \sigma(\kappa y_{\text{base}} \ell(s_i))$
 $\text{PS} \leftarrow 1/(1 + \text{std}(\{\ell(s_i)\}))$
 $\neg s \leftarrow \text{negate}(s)$; $\ell_{\neg} \leftarrow \mathcal{M}.\text{belief_logodds}(\neg s)$; $y_{\neg} \leftarrow \text{sign}(\ell_{\neg})$
 $\text{NS}_{\text{bin}} \leftarrow \mathbb{I}[y_{\neg} = -y_{\text{base}}]$; $\text{NS}_{\text{soft}} \leftarrow \sigma(\kappa \cdot (-y_{\text{base}}) \cdot \ell_{\neg})$
 $\text{NS}_{\text{soft}}^+ \leftarrow \text{NS}_{\text{soft}} \cdot \frac{c_{\text{base}} + \sigma(|\ell_{\neg}|)}{2}$
 $\text{SIMPLE_AKF} \leftarrow 0.5 \text{PC}_{\text{soft}} + 0.5 \text{NS}_{\text{soft}}^+$
 $\text{SIMPLE_AKF}_{\text{stab}} \leftarrow 0.4 \text{PC}_{\text{soft}} + 0.4 \text{NS}_{\text{soft}} + 0.2 \text{PS}$
 Emit node row with all metrics and flags

Preliminary AKF Evaluation Results

Table A.1: Preliminary AKF results across datasets, models, and node polarities. PC = Paraphrase Consistency, NS = Negation Sanity, PS = Paraphrase Stability. Scores closer to 1 indicate stronger consistency or stability.

Dataset	Model	Polarity	#Nodes	PC_{soft}	NS_{soft}	PS	AKF	AKF_{stab}	PC bin	NS bin
MedClaim	Llama	Attack	1273	0.66	0.43	0.77	0.47	0.59	0.72	0.39
MedClaim	Llama	Support	1273	0.85	0.34	0.82	0.55	0.64	0.98	0.18
MedClaim	Mistral	Attack	1273	0.68	0.46	0.80	0.51	0.62	0.87	0.41
MedClaim	Mistral	Support	1273	0.66	0.49	0.78	0.51	0.62	0.88	0.49
MedClaim	Qwen	Attack	1273	0.85	0.39	0.54	0.60	0.60	0.89	0.37
MedClaim	Qwen	Support	1273	0.92	0.58	0.57	0.74	0.71	0.94	0.58
StrategyClaim	Llama	Attack	2290	0.69	0.42	0.80	0.49	0.60	0.76	0.35
StrategyClaim	Llama	Support	2290	0.75	0.40	0.80	0.51	0.62	0.84	0.29
StrategyClaim	Mistral	Attack	2290	0.70	0.43	0.78	0.50	0.61	0.89	0.29
StrategyClaim	Mistral	Support	2290	0.70	0.50	0.79	0.53	0.64	0.88	0.48
StrategyClaim	Qwen	Attack	2290	0.91	0.28	0.55	0.59	0.59	0.93	0.28
StrategyClaim	Qwen	Support	2290	0.94	0.41	0.65	0.65	0.65	0.95	0.41
TruthfulClaim	Llama	Attack	817	0.69	0.43	0.78	0.50	0.60	0.77	0.35
TruthfulClaim	Llama	Support	817	0.81	0.38	0.83	0.54	0.64	0.91	0.25
TruthfulClaim	Mistral	Attack	817	0.69	0.46	0.79	0.51	0.62	0.88	0.42
TruthfulClaim	Mistral	Support	817	0.71	0.55	0.77	0.56	0.66	0.88	0.58
TruthfulClaim	Qwen	Attack	817	0.90	0.36	0.55	0.61	0.61	0.91	0.35
TruthfulClaim	Qwen	Support	817	0.94	0.46	0.68	0.69	0.68	0.96	0.46

The same set of models and datasets were used to evaluate argLLM node faithfulness, which were used for measuring their acceptability and circularity. Table A.1 highlights the results of some preliminary experiments computing the AKF scores for the ArgLLM nodes. Looking closely it can be observed that supporting arguments usually exhibit higher paraphrase consistency (PC) and stability (PS) than attacking arguments and this seems to apply across all datasets, indicating that support nodes are phrased more robustly and less sensitive to perturbations, as they should be. We can also see that the negation sanity (NS) scores are substantially lower than the corresponding

PC scores. This suggests that argument nodes cannot often reverse appropriately under negating conditions, exposing a marked limitation in how LLMs perceive logically opposite polarities.

Overall, AKF complements the SHAKE pipeline with the help of robust and comprehensive node-level proposition checks, and the initial results deem further exploration of this approach, perhaps with better techniques to paraphrase and negate sentences, such as model-assisted restructuring. This would build the case for integrating these tests with the existing suite of faithfulness evaluation metrics and aid the generation of prominent benchmarks.